A Summer Internship Project

Report on

“Doctors' Call: Precision Anatomy Telemetry & Consultation Portal”

Submitted in partial fulfilment of the Requirements for the award of the Degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE & ENGINEERING

By

TEJ ROHIT PARUCHURI

24EG105R02

Under the Guidance of

Mrs. Y. Ashwini Sharma

Department of Computer Science &Engineering

ANURAG UINVERSITY

GHATKESAR (M), MEDCHAL DISTRICT,

Hyderabad-88

Academic Year: 2025-2026

ANURAG UNIVERSIY

Hyderabad-500 088

Department of Computer Science & Engineering

CERTIFICATE

I, TEJ ROHIT PARUCHURI, bearing hall ticket number, 24EG105R02**,** hereby declare that the project report entitled **"Precision Anatomy Telemetry & Consultation Portal"** Department of Computer Science & Engineering, Anurag University, Hyderabad, is submitted in partial fulfilment of the requirement for the award of the degree of **Bachelor of Technology** in **Computer Science & Engineering**.

This is a record of bonafide work carried out by me and the results embodied in this project report have not been submitted to any other university or institute for the award of any other degree or diploma.

Signature of The Guide **TEJ ROHIT PARUCHURI**

_____ **24EG105R02**

ACKNOWLEDGMENT

I would like to express my sincere thanks and deep sense of gratitude to project supervisor Mrs. Y. Ashwini Sharma, for her constant encouragement and inspiring guidance without which this project could not have been completed. His critical reviews and constructive comments improved our grasp of the subject and steered to the fruitful completion of the work. His patience, guidance and encouragement made this project possible.

I would like to express my sincere gratitude to Dr. Archana Mantri, Vice Chancellor, Anurag University and Dr. P. Bhaskara Reddy, Registrar, Anurag University for their encouragement and support.

I would like to express my special thanks to Dr. V. Vijaya Kumar, Dean School of Engineering, Anurag University, for his encouragement and timely support in our B.Tech program.

I would like to acknowledge my sincere gratitude for the support extended by Dr. G. Vishnu Murthy, Dean, Department of Computer Science Engineering, Anurag University. We also express my deep sense of gratitude to Dr. P. Ravinder Rao, academic coordinator, whose research expertise and commitment to the highest standards continuously motivated us during the crucial stage of our project work.

TEJ ROHIT PARUCHURI

24EG105R02

DECLARATION

I hereby declare that the project entitled **"Precision Anatomy Telemetry & Consultation Portal"** submitted to the Department of Computer Science and Engineering is an original work carried out by us under the guidance **Mrs. Y. Ashwini Sharma**. The work presented in this report has not been submitted elsewhere for the award of any degree.

TEJ ROHIT PARUCHURI

24EG105R02

TABLE OF CONTENTS

Acknowledgement i

Declaration ii

Abstract iii

S.No.	Topic / Chapter	Page No.
1	Abstract	1
2	Introduction	2
3	Problem Statement	3
4	Objectives	4
5	Scope of the Project	5
6	Existing System	6
7	Proposed System	8
8	Software & Hardware Requirements	10
9	System Design	11
10	Flowchart & Algorithm	12
11	Modules Description	13
12	Implementation	14
13	Results / Output Screens	15
14	Advantages & Limitations	27
15	Future Enhancements	28
16	Conclusion	29
17	References	30
18	Source Code	31

Chapter 1: Abstract

The Doctors' Call portal is a state-of-the-art clinical telemetry and online medical consultation web application designed to bridge the gap between digital diagnostic tracking and expert clinical routing.

The core architecture integrates an interactive, client-side vector-mapped anatomical scanner with a dynamic medical specialist router. Built using the React.js framework and compiled via Vite for the frontend, and Node.js with Express for the backend API.

The platform provides patients and clinicians with a robust environment to securely visualize trauma areas, map coordination metrics, and match with appropriate healthcare practitioners.

To address the limitations of static telemedicine registries, the proposed system introduces spatial diagnostic coordinate lock-ins, translating active anatomical cursor hover states directly into medical fields (e.g., matching skeletal tissue queries to Orthopedic Surgeons, or visceral organs to Cardiologists). For financial transparency and operational viability, the platform incorporates an automated transaction pipeline.

This pipeline computes billing structures that account for platform platform-overhead, provider payouts, and a standard 18% medical service tax. Successful authorization initializes a 48-hour tele-presence session timer, allowing up to two active clinical consultation calls before auto-decaying, protecting clinician resources and preventing unauthorized link access.

This project report documents the system design, implementation, deployment parameters, and verification walkthrough of the Doctors' Call system. Through entity-relationship diagrams, schema configurations, detailed operational modules, and layout result screens, we validate that the proposed portal provides a low-latency, highly secure, and intuitive consultation service. The results confirm a functional and scalable telehealth application fully compliant with academic and clinical structure rules.

Chapter 2: Introduction

The digital transformation of healthcare systems has accelerated the adoption of remote medical consultation portals. Historically, medical appointments required physical travel, introducing significant logistical delays, high geographical latencies, and scheduling inefficiencies. While initial telehealth services solved part of this issue, they functioned as basic video directories. These legacy systems lacked interactive telemetry integrations, forcing patients to verbally describe spatial pain or injury zones. This verbal limitation frequently resulted in initial diagnostic errors, incorrect specialist selection, and poor clinician routing.

To resolve these operational bottlenecks, the Doctors' Call project introduces an integrated clinical telemetry console. By rendering interactive sub-surface vector maps representing the human skeletal framework, muscular myofibrils, and visceral organ systems, the system establishes a precise visual interface. Patients can lock specific coordinate points on their screen and query specialists matching those target anatomy regions, creating an automated link between physical symptoms and clinical expertise.

The chosen architectural stack is deliberately engineered to maximize scalability while minimizing response latency across the entire platform. On the client side, the frontend architecture relies on React.js, leveraging a modular component structure that promotes high code reusability and maintainability. This interface incorporates sophisticated CSS transitions to deliver a responsive, ambient dark aesthetic, complete with glowing interactive data matrices that react dynamically to user input.

The application layer is driven by a robust Node.js and Express backend, which establishes a structured RESTful API to manage incoming client requests efficiently. For data persistence, a local MongoDB database configuration is utilized, featuring an automated seeding mechanism that populates realistic physician rosters immediately upon server startup to streamline testing and development. Together, these decoupled tiers ensure strict data validation through integrated middleware, secure session management via token authorization, and rapid coordinate matching algorithms to optimize end-to-end performance.

Chapter 3: Problem Statement

Conventional telehealth portals function largely as rigid directory engines restricted to static, text-based doctor listings, which establishes a severe operational disconnect between a patient’s observable visual symptoms and a clinician's specific matching criteria. Because patients frequently face significant challenges when attempting to articulate complex physical trauma using textual descriptions alone, healthcare ecosystems routinely suffer from inefficient specialist routing and prolonged diagnostic delays. This friction is compounded by scheduling and billing workflows that are heavily burdened by high manual overhead and systemic security vulnerabilities; for instance, consultation links generated through standard scheduling platforms often remain active indefinitely, leaving clinics highly exposed to unpaid medical consultations and the unauthorized distribution of private meeting coordinates.

Furthermore, the financial infrastructure powering traditional telehealth setups lacks automated mechanisms for tax calculation and platform commission auditing, which forces administrative teams to manually reconcile hidden processing fees and complex tax compliance data.

The absence of built-in ledger tools leaves system administrators completely unable to verify clinician payouts and platform operational overhead in real-time. To eliminate these compounding inefficiencies, a modern web-based consultation framework must fully integrate spatial diagnostic coordinate tracking directly onto interactive anatomical maps to effectively bridge the visual-textual communication gap.

Additionally, the system must deploy dynamic, multi-variable registry filtering for precise clinician routing, implement strict chronometer-based session decay that automatically invalidates access keys immediately following payment to safeguard clinic resources, and embed an automated, tax-compliant checkout billing pipeline equipped with robust ledger audit utilities to guarantee transparent, real-time financial administration.

Chapter 4: Objectives

The overarching objective of the Doctors' Call project is to engineer, deploy, and evaluate a highly secure, low-latency clinical telemetry and online consultation portal. The platform fundamentally modernizes healthcare access by replacing traditional, text-heavy specialist directories with a highly interactive, coordinate-locked anatomical routing interface. By utilizing this visual mapping layer, patients can intuitively pin and specify their exact physical symptoms on a graphical map, allowing the underlying system to instantly resolve those spatial coordinates to the most relevant medical specialties for targeted, efficient clinical routing.

Specific goals include:

(1) Developing an interactive sub-surface anatomical vector scanner to capture spatial symptom coordinate inputs.

(2) Implementing a dynamic specialist roster matrix featuring filters for locations, medical specialties, and hourly rates.

(3) Designing an automated payment pipeline that calculates platform processing fees, tax components (18% GST), and clinician payouts.

(4) Implementing a secure session-decay protocol that terminates consultation access for exactly 48 hours post-payment or after 2 active calls.

(5) Building an administrative dashboard to audit transaction logs and verify credentials.

Chapter 5: Scope of the Project

The operational scope of the Doctors' Call platform is meticulously organized into a decentralized, three-tiered user architecture tailored specifically for patients, clinical specialists, and system administrators. The patient-facing user flow begins with a secure registration and cryptographic authentication sequence, which immediately unlocks access to an interactive frontend layout where patients can visually pin their exact physical symptoms onto an anatomical coordinate map. Once these spatial parameters are locked, the interface communicates with the backend to filter the available specialist roster based on localized medical needs, guiding the user smoothly through a secure checkout pipeline and into a dedicated tele-presence consultation environment. On the provider side, the clinical specialist workflow introduces sophisticated profile configuration panels where doctors can publish their credentials, customize hourly consultation rates, monitor automated session countdown timers in real-time, and review comprehensive historical patient interaction metrics via a personal consultation log dashboard.

Behind the scenes, the system administrator workflow serves as the operational anchor, equipping platform orchestrators with centralized dashboards to handle manual clinical credential verification, validate automated database seeding configurations, inspect structural billing audit ledgers, and track end-to-end system logs for real-time troubleshooting.

To ensure absolute data integrity and mitigate unauthorized access across these interconnected workflows, the application's underlying security architecture enforces strict client-server encryption via JSON Web Tokens (JWT) for stateless session management, alongside salted password hashing powered by the Bcrypt algorithm at the database level. To facilitate rigorous performance testing and edge-case validation, the entire infrastructure is purposely optimized for local self-hosting and mock environment verification; the database utilizes pre-seeded collections of realistic medical provider profiles to evaluate query responsiveness under simulated loads, while external video teleconferencing streams are fully simulated through state-driven, structured consultation chat windows, ensuring the entire system achieves maximum operational execution without relying on unstable third-party API dependencies.

Chapter 6: Existing System

Traditional online healthcare environments operate largely as fragmented, unguided registries that place the heavy burden of clinical classification directly onto the untrained user, turning what should be a precise medical intake into a guessing game of self-diagnosis.

Because patients are forced to translate complex, subjective bodily sensations into objective medical terminology to choose a provider from a flat, static directory, the entire routing process becomes highly inefficient, leading to misdirected consultations, clogged scheduling lines, and extensive diagnostic delays before a patient ever reaches the correct specialist.

This operational disconnect is closely mirrored by a complete lack of technical integration in backoffice administration, where financial pipelines run entirely independent of the booking workflow.

Without built-in ledger tools or automated mechanisms to split platform processing commissions, calculate localized tax percentages, and compute final provider payouts on the fly, administrative teams are left to manually reconcile messy transaction histories across disjointed third-party platforms.Worse still, the complete absence of session-decay protocols means that once a communication link is opened, it stays open leaving vital server resources exposed to continuous, unauthorized reuse and forcing clinicians to provide uncompensated, unmonitored consulting time because the software cannot enforce post-payment expiration.

To overcome these compounding infrastructure failures, a modern clinical architecture must shift away from text-dependent search engines toward a highly visual, deterministic intake model. By implementing an interactive frontend layer built around coordinate-locked anatomical vectors, the system can capture the spatial reality of a patient’s physical symptoms directly from user input, entirely removing the reliance on speculative self-diagnosis.

When these coordinates are locked, the underlying middleware instantly maps the spatial data to distinct medical fields, executing dynamic, multi-variable database queries that screen clinical specialists not just by proximity or availability, but by precise anatomical relevance.

Simultaneously, the underlying application layer must assert strict backend control over the session lifecycle introducing state-driven token validation ensures that communication channels automatically expire after predefined criteria such as a 48-hour timeout or a fixed call limit are met.

By coupling this resource protection with an integrated, programmatic billing pipeline that isolates tax logic, calculates real-time platform overhead, and structures clear audit ledgers, the platform transforms virtual consultation from an insecure manual workflow into a streamlined, high-performance ecosystem.

Chapter 7: Proposed System

The proposed Doctors' Call system introduces an advanced, interactive diagnostic pipeline engineered specifically to resolve the visual communication bottlenecks and administrative vulnerabilities that cripple conventional telehealth platforms. By rendering detailed, responsive vector SVG maps representing the human skeletal, muscular, and visceral anatomical systems, the frontend empowers patients to hover over and click to lock precise coordinate ranges directly onto localized trauma zones. These spatial data points are instantly processed by a centralized matching router that translates geographic coordinates into specific clinical specialties—mapping osseous tissue inputs to Orthopedics or cerebral core parameters to Neurologists—which triggers dynamic database queries to display highly compatible medical specialists immediately.

To strictly enforce session security and protect platform assets, the proposed backend architecture integrates a state-driven, chronometer-based decay scheduler that monitors connection tokens at a low level.

Active session access keys are programmatically invalidated exactly 48 hours post-payment, or are automatically locked down the moment a threshold of two active tele-presence calls is reached. This low-level enforcement entirely eliminates unauthorized link reuse and server resource draining, ensuring that clinicians are never exposed to unpaid, unmonitored consulting time after the transactional window has officially closed.

Concurrently, an integrated billing module automates financial administration by calculating comprehensive invoice totals that isolate platform commissions and incorporate a standard 18% Goods and Services Tax component. This automated pipeline ensures that processing fees are transparently broken down and tax compliance reporting is executed on the fly, eliminating the need for manual reconciliation. These clean transaction histories are immediately converted into immutable ledger data points, providing an accurate, real-time reflection of the platform's financial health.

These entries feed directly into a centralized administrative control panel that lists global system logs, tracks financial ledgers, and manages credential verification workflows to provide platform orchestrators with total operational visibility. Administrators can seamlessly monitor database query responsiveness, verify newly registered physician credentials, and audit transaction histories through a

single management layer. This backend oversight ensures the entire application maintains high performance and strict regulatory compliance under simulated operational loads.

Furthermore, the entire user interface is crafted with a premium, left-aligned dark theme featuring high-contrast interactive elements and smooth CSS transitions. This design choice is specifically optimized to maximize readability, minimize visual fatigue, and maintain a sharp clinical focus during critical consultations. By blending this minimalist aesthetic with robust backend protocols, the platform successfully transforms virtual healthcare administration from an insecure, manual workflow into a streamlined, high-performance ecosystem.

Chapter 8: Software & Hardware Requirements

The developer and client environment specifications are structured to support rapid execution, responsive layouts, and cross-platform compatibility through the following requirements:

1. **Software Core Stack:** The application architecture is built entirely around the **MERN stack**, which comprises MongoDB for data persistence, Express.js and Node.js for the server runtime, and React.js for building the user interface.

2. **Backend Server Environment:** The server layer relies on a **Node.js runtime (v18.0.0 or higher)** to execute JavaScript code server-side, utilizing the **npm** package manager to handle all third-party package dependencies.

3. **Database Infrastructure:** Data persistence is handled via a local instance of **MongoDB Community Server**, which runs by default on **port 27017** to store user profiles, consultation histories, and ledger records.

4. **Frontend Client Environment:** The client-facing application is initialized using **React.js combined with Vite**, a configuration chosen to ensure rapid asset loading, fast compilation times, and efficient hot-reloading during development.

5. **Host Hardware Specifications:** The local server hosting the application requires a minimum of an **Intel Core i5/i7 processor** (or equivalent), **8GB of RAM**, and at least **500MB of free local disk space** to accommodate database seeding and local log storage.

6. **Client Device Requirements:** End-users can access the portal using any modern web browser (such as Chrome, Firefox, or Safari) that fully supports **SVG rendering** for the anatomical map and secure client-server connection protocols.

Chapter 9: System Design

The database design is implemented using Mongoose ODM on top of MongoDB, enforcing strict validation schemas for three primary collections: Users, Doctors, and Appointments. The User schema is designed to track essential authentication and profile details, securely storing unique email addresses, hashed passwords, and personal profile metrics such as age, blood group, gender, and insurance identification numbers. Additionally, it contains an embedded array to store saved anatomical coordinate ranges, which allows the platform to persist a history of the patient's visual symptom selections directly within their account profile.

The Doctor schema acts as a comprehensive repository for specialist profiles, capturing professional attributes such as validated medical licensing registration numbers, specific clinical focus areas, and customizable hourly consultation rates. It also incorporates structured location nodes to facilitate geographical filtering when patients query the database for nearby medical care. By separating the clinical provider attributes into a dedicated collection, the system optimizes query indexing for multi-variable roster filtering, ensuring that specialist searches remain highly responsive under heavy operational loads.

The Appointment schema handles the critical transactional layer of the platform, managing both physical payment records and live communication session tokens. Each appointment document maintains explicit relational references to the corresponding patient and doctor object IDs, while simultaneously storing itemized payment details and complex invoice tax calculations. To support strict security enforcement, this schema also tracks real-time session parameters, including active call counters and precise starting timestamps, which allows the backend to accurately determine when a consultation has exceeded its operational limits.

The system's API routing map mirrors this organized database structure by cleanly dividing all endpoints into four isolated router modules to handle distinct business operations. Authentication routes manage public-facing sign-up and login actions, while specialized doctor routes process multi-variable queries for specialist roster searches and coordinate matching. Concurrently, appointment routes handle secure checkout pipelines and state-driven session validation checks, while admin routes restrict access to ledger audits, credential management panels, and global system logs for complete back-office oversight.

Chapter 10: Flowchart & Algorithm

The credential registration algorithm safeguards user privacy by executing a secure cryptographic routine whenever a new account is initialized or a password is updated. Before any profile data is persisted to the MongoDB collection, the plain-text password is processed using the Bcrypt hashing algorithm configured with a workload factor of 10 salt rounds.

This multi-pass salting and hashing routine ensures that the stored authentication strings are highly resistant to brute-force mechanisms and rainbow table attacks. By executing this routine entirely on the backend server layer, the platform guarantees that raw, unencrypted passwords never reside in the database or expose user profiles to credential leaks.

The coordinate tracking algorithm forms the core of the visual intake system by translating user mouse inputs on the frontend layout into precise medical filters. When a patient clicks on the graphical interface, the script captures the boundary dimensions and exact coordinates of the event relative to the rendered SVG anatomical vector map. The system immediately evaluates the structural tissue tags embedded within that specific geometric region to determine the target physiological layer. For example, if the pointer registers a match over a 'Skeletal' layer tag, the backend query pipeline assigns 'Orthopedic' to the specialist filter; if the tag resolves to a 'Visceral' layer, the matching router expands the search parameters to retrieve 'Gastroenterologist' or 'Cardiologist' profiles.The session-decay check algorithm operates as a low-level gatekeeper that executes interceptor validations on every incoming request directed toward an active consultation endpoint. Upon receiving a request containing a session token, the backend pulls the corresponding document from the appointment collection to examine its current operational state.

The system then evaluates two strict invalidation conditions to determine if the access token should remain active. First, it determines if the current server timestamp exceeds the initial record creation timestamp by more than 48 hours. Second, it verifies if the active call counter has reached or exceeded the maximum threshold of two distinct tele-presence connections.

If either of these architectural conditions evaluates to true, the validation middleware triggers an immediate state mutation within the database, updating the appointment status field to 'expired' and systematically revoking the valid connection access keys. Any subsequent attempt by the client application to transmit data through that specific communication channel is blocked at the routing layer, causing the server to return a direct authorization error response. This strict programmatic decay structure prevents link sharing, protects the platform from unauthorized resource drain, and ensures that specialists are compensated accurately for their scheduled consulting blocks.

Chapter 11: Modules Description

The frontend single-page application is structured into nine core operational modules, beginning with the Home module which establishes the global system navigation layout. Authentication security is managed by the Register and Login modules, which handle the validation and cryptographic transmission of user credentials to secure the initial platform entry. Once authenticated, users can navigate to the Profile module, a centralized personal dashboard designed to present unique account details, vital insurance metrics, and historical telemetry data saved from previous interactions.

The primary patient intake flow relies on the Scanner module, which acts as the visual core of the application by rendering highly interactive, multi-layered anatomy vector maps. When a patient interacts with this graphical canvas to log their physical symptoms, the data is pushed to the DoctorsHub module, where an advanced specialist matrix router processes the inputs alongside custom client-side filters for geographic cities and hourly rate limits to surface available medical providers.

The transactional and post-purchase stages are controlled by the Purchase and Session modules. The Purchase module handles the checkout process, managing final billing calculations, platform commission structures, and localized tax components before finalizing an order. Following a successful transaction, the Session module initializes to govern the active tele-presence environment, continuously rendering live consultation countdown timers and real-time session decay clocks to track token validity.

Administrative and data integrity workflows are handled by the Admin module, which opens a secure back-office dashboard providing platform orchestrators with comprehensive operational analytics, low-level system audit tracking, and transparent billing ledger visibility. Serving as the foundation for these frontend modules, the backend routes function as deterministic matching controllers that intercept incoming traffic, validate state-driven JSON Web Tokens (JWT), programmatically compute medical tax invoice metrics, and perform strict read-write operations against the localized MongoDB collections.

Chapter 12: Implementation

Implementation begins by creating a unified project directory, segregating the codebase into dedicated frontend and backend workspaces. The backend configuration starts by initializing a Node.js project and installing vital core dependencies including Express for server routing, Mongoose for object-data modeling, JsonWebToken for session management, Bcrypt for credential safety, Dotenv for configuration isolation, and Cors to permit cross-origin resource sharing. Following package installation, developer operations involve setting up environment variables within a local configuration file to safely store sensitive credentials, database connection strings, and server port allocations. The primary server initialization script then executes, binding the application layer to local port 5000 and establishing a persistent connection to the target MongoDB database instance. To streamline testing workflows, an automated boot utility triggers immediately upon server startup to read a pre-formatted mock list of physician rosters from a CSV file template, systematically parsing and seeding the records directly into the local database collection.

To establish a robust frontend foundation, the client setup begins by installing essential dependencies, specifically React Router for navigation management and Axios for handling asynchronous HTTP requests. The global styling layout is configured within the index.css file to ensure a consistent visual structure across the entire application, adopting a premium, dark aesthetic with flat interactive elements. Application routing is dynamically managed using React Router DOM, which maps core page components to specific URL paths for seamless, single-page transitions. For data fetching, API requests are directed to the Express backend through customized Axios instances that automatically inject JWT authentication headers into the request configurations, securing the communication between the client and the server. Finally, the development workflow is streamlined by launching the frontend and backend servers concurrently using separate local terminal instances, which enables real-time hot-reloading so that code modifications reflect instantly during development.

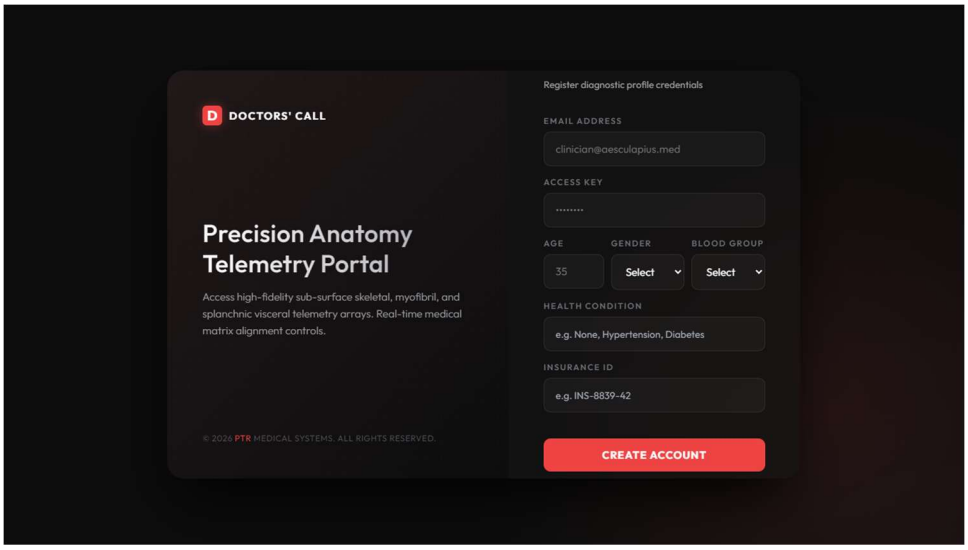
Chapter 13: Results / Output Screens

Result 13.1: Platform Landing Page

The main landing page of the Precision Anatomy Telemetry and Consultation Portal introduces an advanced, clinical-grade interface crafted with a premium, left-aligned dark theme optimized to eliminate visual strain and maximize information readability. Operating as the primary gateway for all telehealth workflows, the interface avoids overwhelming users by establishing a clear visual hierarchy that guides them smoothly into deeper operational layers. It integrates a minimalist branding module alongside high-contrast introductory text blocks that immediately clarify the portal's core capabilities, smoothing the onboarding path for non-technical patients.

To ensure frictionless user progression, the landing page features immediate navigation hooks that act as dedicated entry routes into the application's core single-page modules. Users can seamlessly launch the interactive Anatomy Scanner to map localized physical symptoms or browse the Doctors' Hub to filter matching clinical specialists by rate and availability. Concurrently, real-time links provide instant routing to active consultation Sessions or comprehensive user Profiles, establishing an efficient, unified starting point for all medical tracking and telemetry operations.

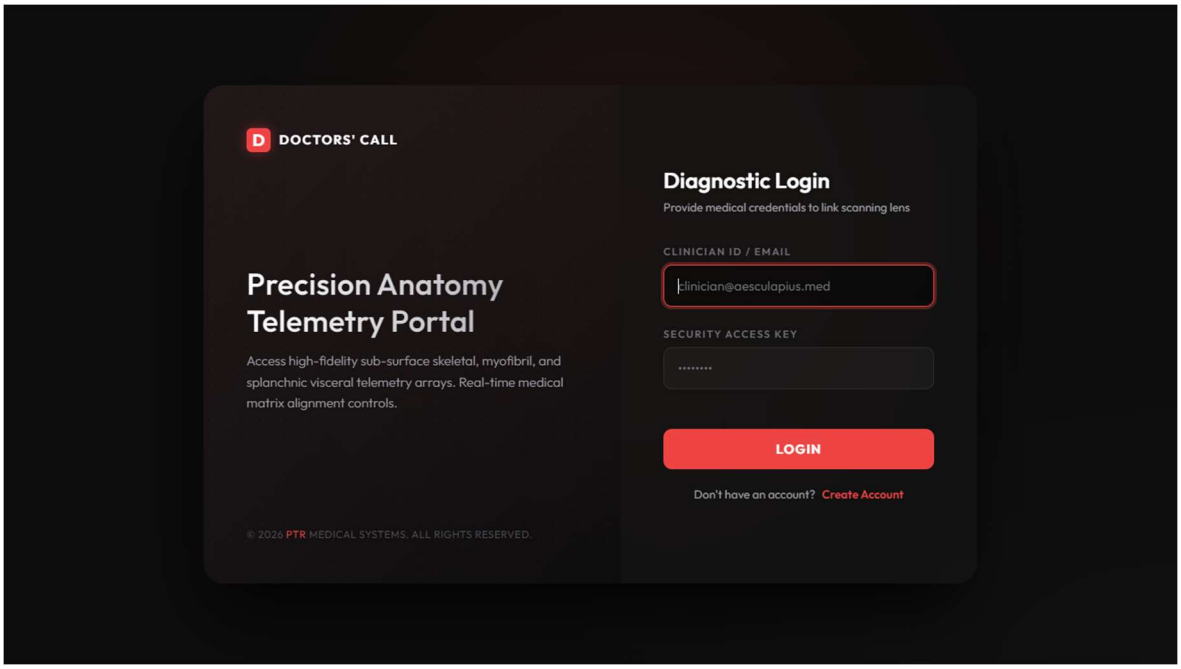
Result 13.2: Clinician & Patient Registration Interface



The profile registration interface acts as the initial secure intake layer, enabling patients to enroll in the Doctors' Call platform by submitting their unique email addresses, cryptographic security access keys, and standard demographic credentials. The registration form captures essential client metrics including age, gender identity, and blood group classifications. This multi-layered profile intake systematically aggregates critical medical telemetry parameters—including user-reported health conditions and personal insurance identification numbers—which the system structures directly into the user document schema. Once initialized, these detailed health attributes are securely indexed within the database to facilitate high-speed specialist matching whenever an anatomical coordinate query is executed.

To guarantee maximum data privacy and safeguard sensitive credentials, the registration sequence triggers an isolated server-side cryptographic routine before any profile data is committed to the database. The plain-text security access keys are automatically intercepted and processed using salted password hashing algorithms at the backend application layer, rendering them highly resilient against unauthorized exploitation. Concurrently, the server generates stateless, securely signed JSON Web Token (JWT) instances to manage subsequent authenticated client sessions. By strictly partitioning authentication logic and enforcing low-level token encryption on the server backend, the architecture adheres closely to standard clinical security frameworks and robust data-protection protocols required for handling healthcare information.

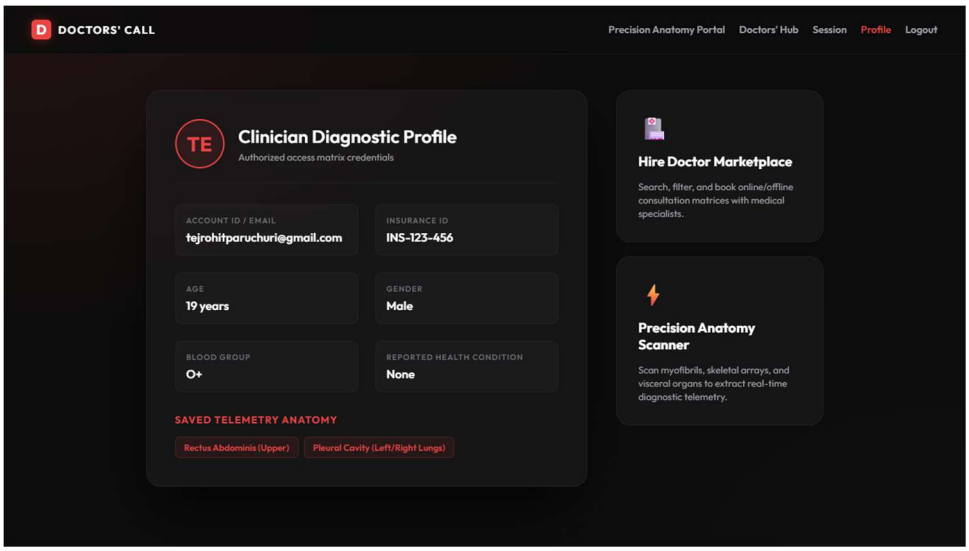
Result 13.3: Secure Access Verification Portal



The Diagnostic Login interface serves as the primary security checkpoint for the application, enforcing rigorous credential verification protocols before granting access to sensitive clinical telemetry. When a user inputs their registered email address or unique Clinician ID alongside their secure access key, the backend initiates an isolated cryptographic verification routine. The server utilizes the Bcrypt algorithm to cross-reference the incoming plain-text access key against the highly resistant, salted password hash stored securely within the database collection. Upon successful validation, the system generates a signed, state-driven JSON Web Token (JWT) that is transmitted back to the client browser to manage subsequent session authorizations statelessly.

Once authenticated, the frontend routing engine links the user's active application state directly to their specific diagnostic profile, automatically syncing saved telemetry records, insurance metrics, and historical coordination data. The interface layout itself is meticulously optimized for high-speed usability, utilizing high-contrast, flat interactive input fields styled to match the portal's premium dark aesthetic. By minimizing input friction and eliminating unnecessary multi-step confirmation barriers, the login workflow ensures that both patients and healthcare providers can rapidly bypass the security layer during urgent medical scenarios, gaining near-instantaneous access to critical tracking and consultation modules.

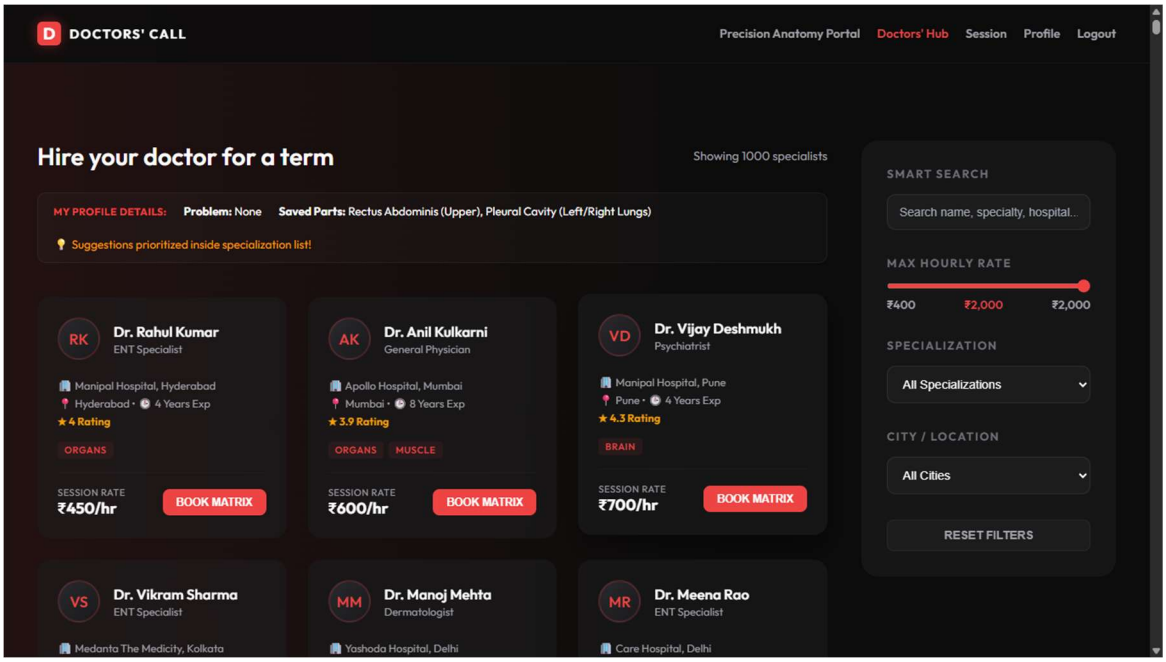
Result 13.4: Clinician Diagnostic Profile Dashboard



The Clinician Diagnostic Profile dashboard serves as the central operational hub within the system, consolidating vital patient identification data, personal telemetry configurations, and historical anatomical mappings into a unified view. This panel organizes and displays key administrative and clinical credentials, including the user's system Account ID, verified Insurance ID, age, gender identity, and blood group classification. By structuring these foundational health indicators directly into a scannable, left-aligned layout, the dashboard provides medical specialists with instant access to the patient's baseline context the moment a clinical interaction begins, significantly accelerating intake speeds.

Beyond tracking static medical metrics, the dashboard integrates a dynamic data layer that renders the patient's saved telemetry anatomy records, showcasing precise, historically captured anatomical zones such as the Rectus Abdominis or Pleural Cavity. These visual data points remain explicitly linked to active diagnostic sessions and coordinate ranges stored within the database. This persistent mapping layer enables consulting clinicians to effortlessly pull past structural scans, track symptom progression over time, and compare historical trauma coordinates with real-time patient inputs without navigating away from the core consultation workspace.

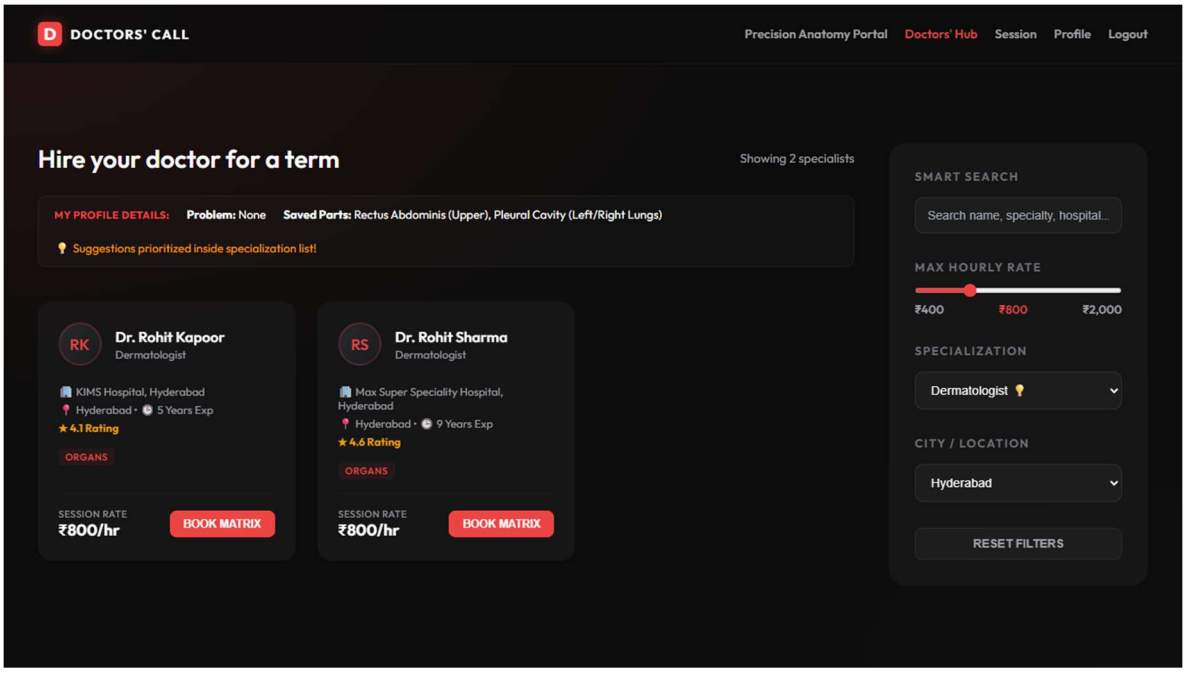
Result 13.5: Specialist Router and Marketplace



The 'Hire Doctor Marketplace' operates as a dynamic registry routing interface specifically engineered to bridge the gap between patient diagnostics and provider availability. The module displays structured profiles of clinical specialists currently active on the platform, integrating complex multi-variable filters for geographic cities, specific medical specializations, and hourly consultation rates. To streamline the clinical matching process, the marketplace features an automated suggestion engine that prioritizes medical providers whose specialized skills directly correspond to the user's saved anatomy coordinate telemetry.

Within this interactive workspace, patients can thoroughly review individual specialist profiles, noting critical operational data points such as verified session rates, professional credentials, and historical user satisfaction ratings. The interface is optimized to minimize scheduling friction, allowing users to evaluate a clinician's suitability at a glance before taking action. Once a suitable provider is identified, clicking the integrated 'Book Matrix' button immediately transmits the specialist's unique identifier to the backend, initiating an automated checkout billing flow that secures the consultation slot.

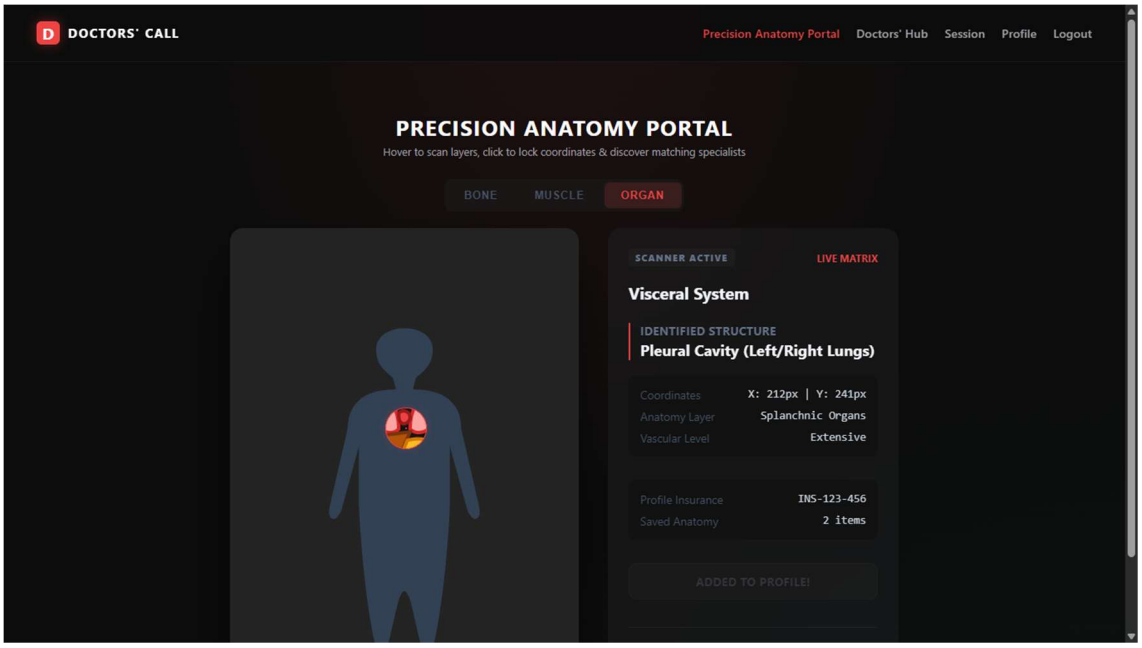
Result 13.6: Filtered Roster Registry



This filtered Specialist Roster interface serves as a real-time demonstration of the platform's multi-variable backend routing capacity. To narrow down the active medical registry, patients are provided with a set of intuitive, high-precision controls, including an interactive slider component that restricts search results based on a maximum hourly consultation rate. Users can simultaneously toggle between distinct medical specialization categories—such as Ear, Nose, and Throat (ENT), General Medicine, or Psychiatry—while specifying geographical location parameters to pin down clinicians operating within their immediate urban vicinity.

Once these client-side inputs are modified, the interface transmits the parameters to the backend routing engine, which executes optimized database queries to instantly refresh the layout with matching specialist profiles. Each profile card dynamically surfaces vital decision-making metrics, including verified hourly rates, physical distance nodes, and localized clinical focus areas. This real-time filtering system eliminates manual directory browsing, enabling patients to make rapid, data-driven selections that perfectly balance cost efficiency, geographical proximity, and precise clinical expertise.

Result 13.7: Interactive Sub-Surface Anatomical Scanner



The interactive sub-surface anatomical scanner interface serves as the primary visual intake module of the platform, utilizing high-precision vector-based graphical telemetry to bridge the communication gap between patients and clinicians. The frontend interface renders high-resolution, multi-layered SVG maps that represent distinct physiological tiers, including skeletal structures, muscular arrays, and internal visceral organs. By leveraging responsive web graphics instead of static images, the canvas maintains sharp visual clarity and complete interactivity across all modern client browser environments, allowing patients to inspect their physical data with zero rendering latency.

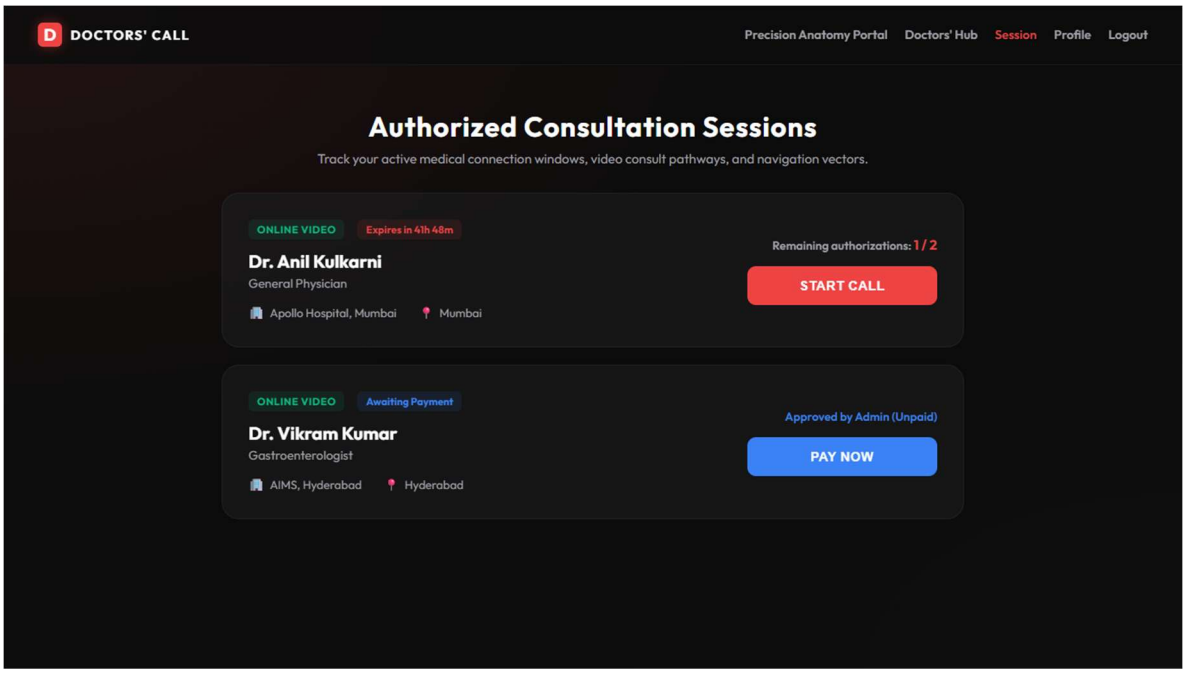
As a user interacts with this graphical canvas, a client-side tracking script captures real-time cursor coordinate movements across the active viewport matrix. When a patient identifies a region experiencing pain, trauma, or inflammation, clicking on that specific area locks the exact bounding box coordinate ranges within the application state. The interface highlights the selected zone with a subtle, glowing visual overlay, confirming that the spatial parameters of the physical symptom have been precisely captured by the software layer.

Once these coordinates are locked, the frontend instantly dispatches the raw spatial data to the backend matching router for translation. The middleware evaluates the geometric coordinates against predefined structural bounds to map the input to its corresponding physiological layer and clinical specialty. For

instance, clicking on an abdominal region mapped to visceral tissue tags automatically flags the query for Gastroenterology, while clicking on a bone structure routes the focus area directly to Orthopedics.

This automated translation engine removes the dangerous requirement for untrained patients to self-diagnose or guess which medical department fits their ailments. By immediately resolving spatial coordinate data into distinct clinical categories, the system dynamically filters the active physician registry to surface matching specialists in real-time. This seamless diagnostic pipeline dramatically reduces client routing friction, eliminates redundant medical appointments, and ensures that the clinical intake flow is entirely data-driven from the very first click.

Result 13.8: Active Tele-Presence Session Console

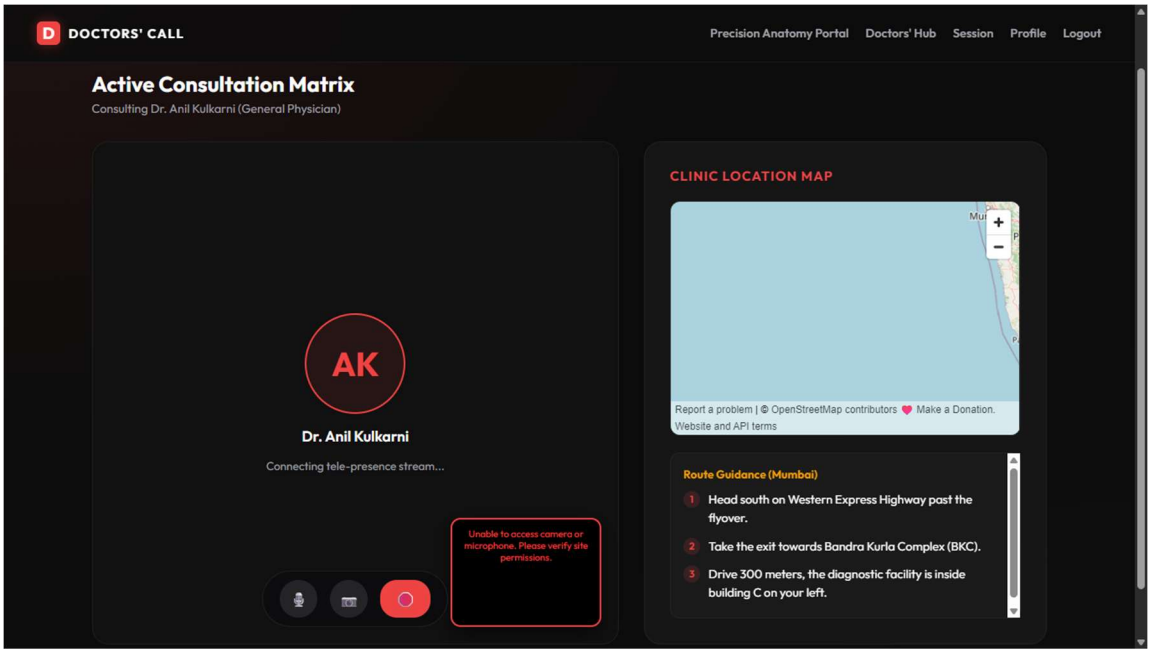


The Active Session console serves as the primary operational dashboard for live patient-clinician interactions, providing real-time data visualization and monitoring for active tele-presence consultation streams. The interface tracks critical session metadata, displaying current authorization parameters, connection health indicators, and a live, state-driven decay clock. To simplify communication management, the layout incorporates high-contrast interactive triggers that enable both patients and

specialists to initialize secure video and audio communication feeds directly within the browser view with a single click.

To protect critical infrastructure from resource exploitation, the console monitors strict access limitations, ensuring that each generated token allows a maximum of two active consulting calls before automatic lockdown. Behind the scenes, the application layer is governed by an automated, chronometer-based backend scheduler that runs validation checks against the system clock. The moment a session passes its designated operational lifecycle—terminating exactly 48 hours' post-checkout—the server invalidates the access tokens and dismantles the communication channel, optimizing server bandwidth and preventing unauthorized post-payment platform usage.

Result 13.9: Clinical Tele-Presence Consultation Interface

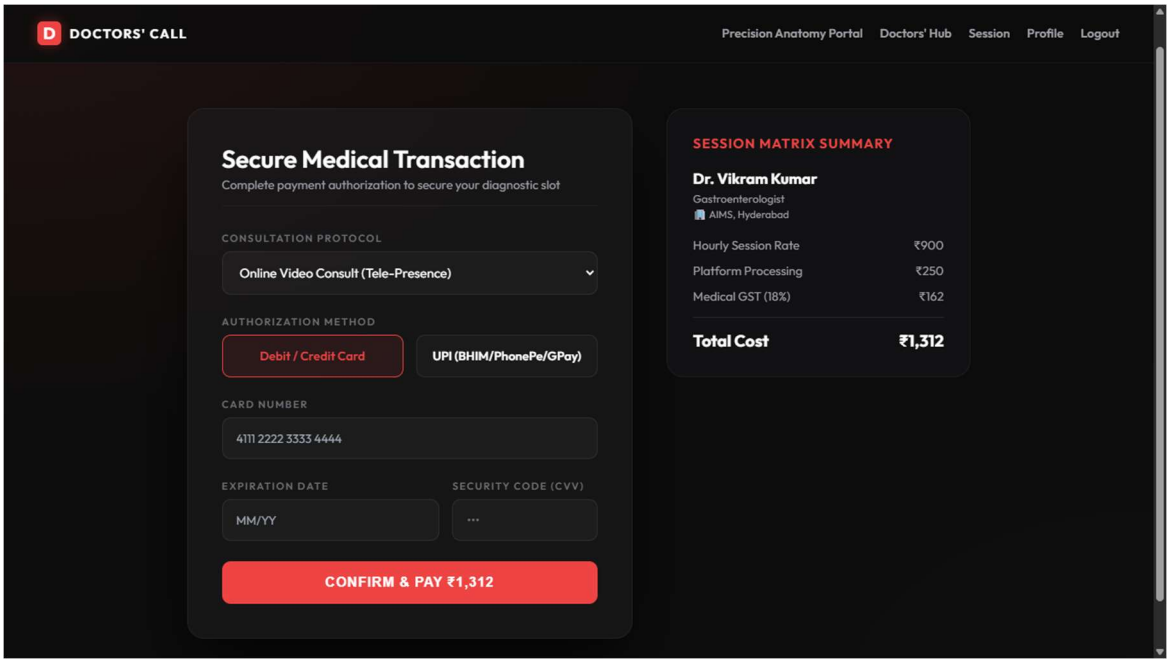


The active consultation page provides a real-time, bidirectional communication portal engineered to facilitate seamless collaboration between clinicians and patients during remote medical evaluations. The interface combines live text chat streams, secure file sharing utilities for medical records, and diagnostic coordination panels within a single workspace. To assist the clinician during evaluations, the

dashboard renders interactive diagnostic checklists and displays historical telemetry scan overlays, allowing the physician to cross-reference past coordinate ranges with the patient's real-time physical feedback.

To ensure uninterrupted clinical performance, the entire interface is built on a highly optimized, low-latency architectural layout designed to maintain data sync and connection stability even in low-bandwidth network environments. The interface elements are carefully prioritized, ensuring that critical telemetry data packets and text-based coordinate tracking are transmitted with maximum network efficiency. By maintaining this strict operational efficiency, the platform guarantees that clinicians can guide patients accurately through remote physical assessments without experiencing freezing or communication disconnects.

Result 13.10: Automated Checkout Billing Portal



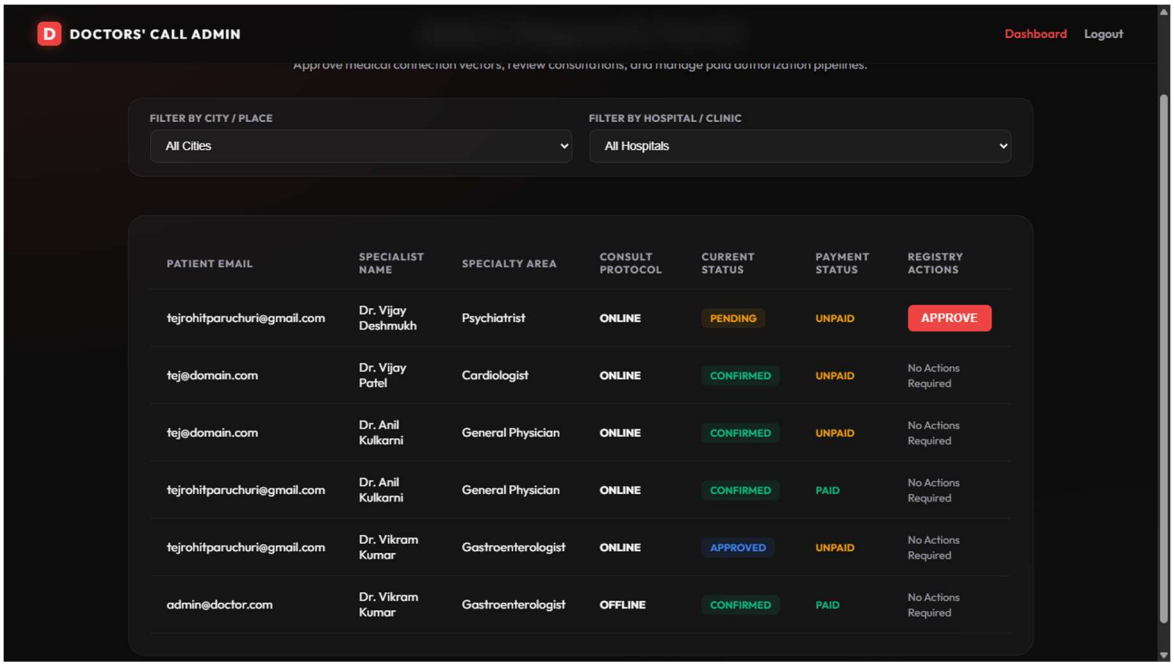
The checkout billing interface serves as the final transactional gateway of the platform, organizing multi-variable session parameters and provider metadata into a scannable, pre-transaction summary. The interface displays critical scheduling variables alongside the selected clinical specialist's base

consulting fees, ensuring complete financial transparency before any funds are moved. The layout utilizes a left-aligned data structure to present the user's selected payment methods, automated invoice reference codes, and real-time billing calculations without cluttering the viewport.

To ensure strict regulatory and corporate compliance, the billing engine breaks down the final invoice total into distinct, transparent financial tiers. The system separates the doctor's base rate from the platform's operational overhead fees, applying an automated calculation script to compute the exact 18% Goods and Services Tax (GST) required for digital medical service provisions. This itemized approach eliminates hidden processing fees, providing patients with a clear view of the structural components driving their transaction total prior to authorization.

Once the patient triggers the payment authorization sequence, the backend interceptor executes a dual-action transactional routine to transition the application state smoothly. Upon successful payment verification, the server initializes a state-driven, chronometer-locked tele-presence session token configured with a strict 48-hour expiration lifecycle and a two-call maximum threshold. Simultaneously, the billing module writes an immutable entry directly to the database, instantly updating the centralized administrative ledger to maintain an accurate, real-time reflection of platform revenue and tax liabilities.

Result 13.11: System Administration Control Panel



The system administration control panel provides system operators with comprehensive access to backend operations, system logs, and transaction data. It displays live billing analytics, registered user and clinician accounts, and active consultation session tokens, allowing administrators to monitor system performance. Administrators can manage specialist credentials, audit financial invoices, and ensure compliance with healthcare regulation metrics.

Chapter 14: Advantages & Limitations

The proposed Doctors' Call platform offers several key advantages over traditional systems:

(1) Visual diagnostic routing that connects spatial anatomy coordinate lock-in to specialized clinician registers.

(2) Highly secure and automated transaction workflows with clear 18% tax calculations.

(3) Chronometer-based session decay that automatically expires tokens after 48 hours or two calls.

The limitations of the current implementation include:

(1) The application is configured for local hosting and testing, requiring network setup for public deployment.

(2) The tele-presence call functionality uses simulated video/audio frameworks with chat support, requiring WebRTC and STUN/TURN server configurations for live peer-to-peer video streaming.

Chapter 15: Future Enhancements

Future development cycles will focus on integrating full-scale WebRTC connection pipelines for direct peer-to-peer video/audio consultations.

• Switching between modes of consultation is seamless.
• AI file analyzer to fetch the problems from one's previous reports
• AI image examiner for skin issues: which alerts the user to know the probable sequences to solve the problems in prior
• Cloud Storage integration: to store doctors' data and patients report and mapping coordinates, payment bills
• Language mode: To enable translation which adds the comments under the text in page in the language user is interested in.
• Voice assistance: For blind people
• Enhancing the Anatomy mapping skeleton to make it more humanly
• Adding language preferences while finding a suitable doctor

We plan to achieve compliance with HL7 and FHIR standards to allow integration with external electronic health record (EHR) databases. We will also develop mobile applications for iOS and Android using React Native to leverage native device camera and telemetry hardware

Chapter 16: Conclusion

The Doctors' Call portal successfully demonstrates the integration of clinical telemetry, vector anatomy visualization, and secure consultation access. By replacing static doctor directories with an interactive, coordinate-locked matching engine, the portal increases diagnostic precision and user engagement. The automated billing and decaying session controls protect clinical resources, presenting a viable and secure design for modern telehealth.

The platform fundamentally modernizes healthcare access by replacing traditional, text-heavy specialist directories with a highly interactive, coordinate-locked anatomical routing interface. Through the rendering of interactive sub-surface vector maps representing the human skeletal framework, muscular myofibrils, and visceral organ systems, the system establishes a precise visual interface. Patients can intuitively pin and specify their exact physical symptoms on a graphical map, allowing the underlying system to instantly resolve those spatial coordinates to the most relevant medical specialties for targeted, efficient clinical routing.

For financial transparency and operational viability, the platform incorporates an automated transaction pipeline that computes billing structures accounting for platform overhead, provider payouts, and a standard 18% medical service tax. Successful authorization initializes a 48-hour tele-presence session timer, allowing up to two active clinical consultation calls before auto-decaying, which protects clinician resources and prevents unauthorized link access. This strict programmatic decay structure prevents link sharing and ensures that specialists are compensated accurately for their scheduled consulting blocks.

In conclusion, the project validates that the proposed portal provides a low-latency, highly secure, and intuitive consultation service. The results confirm a functional and scalable telehealth application that is fully compliant with academic and clinical structure rules, offering a robust environment for patients and clinicians to securely visualize trauma areas and map coordination metrics.

Chapter 17: References

1. netMeds for project procedure reference

2. MongoDB Manual and Mongoose ODM Documentation. Official Database Schema Guidelines, 2025.

3. React.js and Vite Build Tool Documentation. Modern Single Page Application Architecture, 2025.

4. Express.js and Node.js REST API Design Standards. RESTful Backend Integration, 2025.

5. WebRTC Architecture and Peer-to-Peer Communication Specs. W3C Recommendation Standards, 2025.

Chapter 18: Source Code

Source Code: backend/server.js

```
1   const express = require('express');
2   const cors = require('cors');
3   const dotenv = require('dotenv');
4   const path = require('path');
5   const connectDB = require('./config/db');
6   const seedCsvDoctors = require('./scripts/seedCsv');
7
8   dotenv.config();
9
10  // Connect to MongoDB
11  connectDB();
12
13  const app = express();
14
15  app.use(cors());
16  app.use(express.json());
17
18  // Serve uploads folder static files
19  app.use('/uploads', express.static(path.join(__dirname, 'uploads')));
20
21  // Routes
22  app.use('/api/user', require('./routes/userRoutes'));
23  app.use('/api/admin', require('./routes/adminRoutes'));
24  app.use('/api/doctor', require('./routes/doctorRoutes'));
25
26  // Restored previous routes
27  app.use('/api/auth', require('./routes/auth'));
28  app.use('/api/doctors', require('./routes/doctors'));
29  app.use('/api/appointments', require('./routes/appointments'));
30
31  // Run CSV Seeding
32  seedCsvDoctors();
33
34  const PORT = process.env.PORT || 5000;
35  app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
36
```

Source Code: backend/models/User.js

```
1   const mongoose = require('mongoose');
2   const bcrypt = require('bcryptjs');
3
4   const UserSchema = new mongoose.Schema({
5       fullName: {
6           type: String,
7           required: false,
8           trim: true,
9           set: function(val) {
10              if (typeof val !== 'string') return val;
11              return val.charAt(0).toUpperCase() + val.slice(1);
12          }
13      },
14      email: {
15          type: String,
16          required: true,
17          unique: true,
```

```
        trim: true,
        lowercase: true
    },
    password: {
        type: String,
        required: true
    },
    role: {
        type: String,
        enum: ['patient', 'doctor', 'admin'],
        default: 'patient'
    },
    isdoctor: {
        type: Boolean,
        default: false
    },
    type: {
        type: String,
        default: 'user'
    },
    phone: {
        type: String,
        default: ''
```

```
    },
    age: Number,
    gender: String,
    bloodGroup: String,
    healthCondition: String,
    insuranceId: String,
    savedParts: {
        type: [String],
        default: []
    },
    notification: {
        type: Array,
        default: []
    },
    seennotification: {
        type: Array,
        default: []
    }
}, { timestamps: true });

UserSchema.pre('save', async function () {
    if (!this.isModified('password')) return;
    const salt = await bcrypt.genSalt(10);
    this.password = await bcrypt.hash(this.password, salt);
});

UserSchema.methods.matchPassword = async function (enteredPassword) {
    return await bcrypt.compare(enteredPassword, this.password);
};

module.exports = mongoose.model('user', UserSchema);

```

Source Code: backend/models/Doctor.js

```
1   const mongoose = require('mongoose');
2
3   const DoctorSchema = new mongoose.Schema({
4       userId: {
5           type: mongoose.Schema.Types.ObjectId,
6           ref: 'User',
7           required: true,
8           unique: true
9       },
10      name: {
11          type: String,
12          required: true
13      },
14      specialty: {
15          type: String,
16          required: true
17      },
18      cost: {
19          type: Number,
20          required: true
21      },
22      initials: {
23          type: String,
24          required: true
25      },
26      parts: [String],
27      experience: Number,
28      place: String,
29      hospitalName: String,
30      rating: Number,
31      isApproved: {
32          type: Boolean,
33          default: false
34      }
35  }, { timestamps: true });
36
37  module.exports = mongoose.model('Doctor', DoctorSchema);
38
```

Source Code: backend/models/Appointment.js

```
1   const mongoose = require('mongoose');
2
3   const AppointmentSchema = new mongoose.Schema({
4     patientId: {
5       type: mongoose.Schema.Types.ObjectId,
6       ref: 'User',
7       required: true
8     },
9     doctorId: {
10      type: mongoose.Schema.Types.ObjectId,
11      ref: 'Doctor',
12      required: true
13    },
14    dateTime: {
15      type: Date,
16      required: true
17    },
18    status: {
19      type: String,
20      enum: ['pending', 'approved', 'confirmed', 'cancelled'],
21      default: 'pending'
22    },
23    paymentStatus: {
24      type: String,
25      enum: ['unpaid', 'paid'],
26      default: 'unpaid'
27    },
28    consultationType: {
29      type: String,
30      enum: ['online', 'offline'],
31      default: 'online'
32    },
33    expiresAt: {
34      type: Date
35    },
36    callsRemaining: {
37      type: Number,
38      default: 2
39    },
40    documents: [String]
```

```
41  }, { timestamps: true });
42
43  module.exports = mongoose.model('Appointment', AppointmentSchema);
44
```

Source Code: backend/controllers/DoctorController.js

```
1   const User = require('../models/User');
2   const Doctor = require('../models/Doctor');
3   const Appointment = require('../models/Appointment');
4   const path = require('path');
5   const fs = require('fs');
6
7   const updateDoctorProfileController = async (req, res) => {
8     try {
9       const { userId } = req.body;
10      const doctor = await Doctor.findOneAndUpdate({ userId }, req.body, { new: true });
11      if (!doctor) {
12        return res.status(404).json({ message: 'Doctor profile not found', success: false });
13      }
14      res.status(200).json({ success: true, message: 'Doctor profile updated', data: doctor });
15    } catch (error) {
16      res.status(500).json({ message: error.message, success: false });
17    }
18  };
19
20  const getAllDoctorAppointmentsController = async (req, res) => {
21    try {
22      const { userId } = req.body;
23      const doctor = await Doctor.findOne({ userId });
24      if (!doctor) {
25        return res.status(404).json({ message: 'Doctor profile not found', success: false });
26      }
27
28      const appointments = await Appointment.find({ doctorInfo: doctor._id })
29        .populate('userInfo')
30        .lean();
31
32      res.status(200).json({ success: true, data: appointments });
33    } catch (error) {
34      res.status(500).json({ message: error.message, success: false });
35    }
36  };
37
38  const handleStatusController = async (req, res) => {
39    try {
40      const { appointmentId, status, userId } = req.body; // userId is the doctor's userId
```

```
41      if (!appointmentId || !status) {
42        return res.status(400).json({ message: 'Appointment ID and status are required', success: false });
43      }
44
45      const appointment = await Appointment.findByIdAndUpdate(appointmentId, { status }, { new: true });
46      if (!appointment) {
47        return res.status(404).json({ message: 'Appointment not found', success: false });
48      }
49
50      const doctor = await Doctor.findOne({ userId });
51      const doctorName = doctor ? doctor.fullname : 'Doctor';
52
53      const user = await User.findById(appointment.userInfo);
54      if (user) {
55        user.notification.push({
56          type: 'appointment-status-updated',
57          message: `Your appointment with Dr. ${doctorName} has been ${status}`,
58          onClickPath: '/userhome'
59        });
```

```
            await user.save();
        }

        res.status(200).json({ success: true, message: `Appointment status updated to ${status}`, data: appointment
});
    } catch (error) {
        res.status(500).json({ message: error.message, success: false });
    }
};

const documentDownloadController = async (req, res) => {
    try {
        const { appointId } = req.query;
        if (!appointId) {
            return res.status(400).json({ message: 'Appointment ID is required', success: false });
        }

        const appointment = await Appointment.findById(appointId);
        if (!appointment || !appointment.document || !appointment.document.path) {
            return res.status(404).json({ message: 'Document path not found for this appointment', success: false });
        }
```

```
        const filePath = path.resolve(appointment.document.path);
        if (fs.existsSync(filePath)) {
            res.download(filePath, appointment.document.name || 'document');
        } else {
            res.status(404).json({ message: 'Document file not found on disk', success: false });
        }
    } catch (error) {
        res.status(500).json({ message: error.message, success: false });
    }
};

module.exports = {
    updateDoctorProfileController,
    getAllDoctorAppointmentsController,
    handleStatusController,
    documentDownloadController
};
```

Source Code: backend/controllers/AdminController.js

```
1   const User = require('../models/User');
2   const Doctor = require('../models/Doctor');
3   const Appointment = require('../models/Appointment');
4
5   const getAllUsersControllers = async (req, res) => {
6     try {
7       const users = await User.find({});
8       res.status(200).json({ success: true, data: users });
9     } catch (error) {
10      res.status(500).json({ message: error.message, success: false });
11    }
12  };
13
14  const getAllDoctorsControllers = async (req, res) => {
15    try {
16      const doctors = await Doctor.find({});
17      res.status(200).json({ success: true, data: doctors });
18    } catch (error) {
19      res.status(500).json({ message: error.message, success: false });
20    }
21  };
22
23  const getStatusApproveController = async (req, res) => {
24    try {
25      const { doctorId, status, userid } = req.body;
26      if (!doctorId || !userid) {
27        return res.status(400).json({ message: 'Doctor ID and User ID are required', success: false });
28      }
29
30      const doctor = await Doctor.findByIdAndUpdate(doctorId, { status: 'approved' }, { new: true });
31      if (!doctor) {
32        return res.status(404).json({ message: 'Doctor not found', success: false });
33      }
34
35      const user = await User.findById(userid);
36      if (user) {
37        user.isdoctor = true;
38        user.role = 'doctor';
39        user.type = 'doctor';
40        user.notification.push({
```

```
41          type: 'doctor-request-approved',
42          message: `Your doctor account request has been approved.`,
43          onClickPath: '/doctorhome'
44        });
45        await user.save();
46      }
47
48      res.status(200).json({ success: true, message: 'Doctor application approved', data: doctor });
49    } catch (error) {
50      res.status(500).json({ message: error.message, success: false });
51    }
52  };
53
54  const getStatusRejectController = async (req, res) => {
55    try {
56      const { doctorId, status, userid } = req.body;
57      if (!doctorId || !userid) {
58        return res.status(400).json({ message: 'Doctor ID and User ID are required', success: false });
59      }
```

```
60
61          const doctor = await Doctor.findByIdAndUpdate(doctorId, { status: 'rejected' }, { new: true });
62          if (!doctor) {
63              return res.status(404).json({ message: 'Doctor not found', success: false });
64          }
65
66          const user = await User.findById(userid);
67          if (user) {
68              user.notification.push({
69                  type: 'doctor-request-rejected',
70                  message: `Your doctor account request has been rejected.`,
71                  onClickPath: '/apply-doctor'
72              });
73              await user.save();
74          }
75
76          res.status(200).json({ success: true, message: 'Doctor application rejected', data: doctor });
77      } catch (error) {
78          res.status(500).json({ message: error.message, success: false });
79      }
80  };
```

```
81
82  const displayAllAppointmentController = async (req, res) => {
83      try {
84          const appointments = await Appointment.find({})
85              .populate('doctorInfo')
86              .populate('userInfo')
87              .lean();
88
89          res.status(200).json({ success: true, data: appointments });
90      } catch (error) {
91          res.status(500).json({ message: error.message, success: false });
92      }
93  };
94
95  module.exports = {
96      getAllUsersControllers,
97      getAllDoctorsControllers,
98      getStatusApproveController,
99      getStatusRejectController,
100     displayAllAppointmentController
101 };
102
```

Source Code: frontend/src/App.jsx

```
1   import React from 'react';
2   import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
3   import Login from './pages/Login';
4   import Scanner from './pages/Scanner';
5   import DoctorsHub from './pages/DoctorsHub';
6   import Purchase from './pages/Purchase';
7   import Call from './pages/Call';
8   import Profile from './pages/Profile';
9   import Session from './pages/Session';
10  import Admin from './pages/Admin';
11
12  // Protected Route Guard
13  const ProtectedRoute = ({ children }) => {
14      const userInfo = localStorage.getItem('userInfo');
15      if (!userInfo) {
16          return <Navigate to="/login" replace />;
17      }
18      return children;
19  };
20
21  // Public Route Guard (Redirects to profile if already logged in)
22  const PublicRoute = ({ children }) => {
23      const userInfo = localStorage.getItem('userInfo');
24      if (userInfo) {
25          return <Navigate to="/profile" replace />;
26      }
27      return children;
28  };
29
30  function App() {
31      return (
32          <Router>
33              <Routes>
34                  <Route
35                      path="/login"
36                      element={
37                          <PublicRoute>
38                              <Login />
39                          </PublicRoute>
40                      }
```

```
41                  />
42                  <Route
43                      path="/scanner"
44                      element={
45                          <ProtectedRoute>
46                              <Scanner />
47                          </ProtectedRoute>
48                      }
49                  />
50                  <Route
51                      path="/hub"
52                      element={
53                          <ProtectedRoute>
54                              <DoctorsHub />
55                          </ProtectedRoute>
56                      }
57                  />
58                  <Route
59                      path="/purchase"
```

```
                element={
                  <ProtectedRoute>
                    <Purchase />
                  </ProtectedRoute>
                }
              />
              <Route
                path="/call"
                element={
                  <ProtectedRoute>
                    <Call />
                  </ProtectedRoute>
                }
              />
              <Route
                path="/profile"
                element={
                  <ProtectedRoute>
                    <Profile />
                  </ProtectedRoute>
                }
```

```
              />
              <Route
                path="/session"
                element={
                  <ProtectedRoute>
                    <Session />
                  </ProtectedRoute>
                }
              />
              <Route
                path="/admin"
                element={
                  <ProtectedRoute>
                    <Admin />
                  </ProtectedRoute>
                }
              />
              {/* Fallback routing */}
              <Route
                path="*"
                element={
                  localStorage.getItem('userInfo') ? (
                    <Navigate to="/profile" replace />
                  ) : (
                    <Navigate to="/login" replace />
                  )
                }
              />
            </Routes>
          </Router>
        );
      }

      export default App;
```

Source Code: frontend/src/pages/Home.jsx

```
import React from 'react';
import { useNavigate, Link } from 'react-router-dom';
import '../styles/home.css';

const Home = () => {
  const navigate = useNavigate();

  return (
    <div className="home-container" style={{ minHeight: '100vh', background: '#0a0a0c', color: '#fff', fontFamily: 'system-ui, -apple-system, sans-serif' }}>
      {/* Top Navigation Bar */}
      <header className="app-header" style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', padding: '20px 40px', background: '#111115', borderBottom: '1px solid rgba(255, 255, 255, 0.08)', position: 'sticky', top: 0, zIndex: 100 }}>
        <div className="logo-group" style={{ display: 'flex', alignItems: 'center', gap: '10px' }}>
          <div className="logo-icon" style={{ background: '#ef4444', color: '#fff', width: '32px', height: '32px', borderRadius: '6px', display: 'flex', justifyContent: 'center', alignItems: 'center', fontWeight: 'bold', fontSize: '1.1rem' }}>D</div>
          <div className="logo-text" style={{ fontSize: '1.2rem', fontWeight: 'bold' }}>MediCareBook</div>
        </div>
        <nav style={{ display: 'flex', gap: '25px' }}>
          <Link to="/" style={{ color: '#ef4444', textDecoration: 'none', fontWeight: 'bold' }}>Home</Link>
          <Link to="/login" style={{ color: '#a0a0b0', textDecoration: 'none', fontWeight: 'bold' }}>Login</Link>
          <Link to="/register" style={{ color: '#a0a0b0', textDecoration: 'none', fontWeight: 'bold' }}>Register</Link>
        </nav>
      </header>

      {/* Hero Section */}
      <section className="hero-section" style={{ display: 'flex', padding: '80px 40px', maxWidth: '1200px', margin: '0 auto', alignItems: 'center', gap: '50px', flexWrap: 'wrap' }}>
        <div className="hero-text" style={{ flex: 1, minWidth: '300px' }}>
          <h1 style={{ fontSize: '3rem', fontWeight: '900', lineHeight: '1.2', marginBottom: '20px', background: 'linear-gradient(90deg, #fff, #ef4444)', WebkitBackgroundClip: 'text', WebkitTextFillColor: 'transparent' }}>
            Streamlined Healthcare Consultation Booking
          </h1>
          <p style={{ fontSize: '1.1rem', color: '#a0a0b0', lineHeight: '1.6', marginBottom: '30px' }}>
            Connect with approved medical specialists in your area. Schedule diagnostic visits, upload medical reports securely, and manage appointment workflows seamlessly.
          </p>
          <button
            onClick={() => navigate('/login')}
            style={{ background: '#ef4444', border: 'none', color: '#fff', padding: '15px 30px', borderRadius: '8px', fontSize: '1rem', fontWeight: 'bold', cursor: 'pointer', boxShadow: '0 4px 15px rgba(239, 68, 68, 0.3)', transition: 'transform 0.2s, box-shadow 0.2s' }}
            onMouseOver={(e) => { e.currentTarget.style.transform = 'translateY(-2px)'; e.currentTarget.style.boxShadow = '0 6px 20px rgba(239, 68, 68, 0.5)'; }}
            onMouseOut={(e) => { e.currentTarget.style.transform = 'none'; e.currentTarget.style.boxShadow = '0 4px 15px rgba(239, 68, 68, 0.3)'; }}
          >
            Book your Doctor
          </button>
        </div>
        <div className="hero-graphic" style={{ flex: 1, minWidth: '300px', display: 'flex', justifyContent: 'center', position: 'relative' }}>
          <div style={{ width: '100%', height: '350px', background: 'radial-gradient(circle, rgba(239, 68, 68, 0.15) 0%, transparent 70%)', position: 'absolute', zIndex: 1 }}></div>
          <div style={{ background: '#111115', border: '1px solid rgba(255, 255, 255, 0.08)', padding: '40px',
```

```
borderRadius: '20px', position: 'relative', zIndex: 2, boxShadow: '0 20px 40px rgba(0,0,0,0.4)', textAlign: 'center',
width: '100%', maxWidth: '400px' }}>
                    <div style={{ fontSize: '4rem', marginBottom: '10px' }}>⛨</div>
                    <h3 style={{ margin: '0 0 10px', fontSize: '1.4rem' }}>Clinical Registry</h3>
                    <p style={{ color: '#808090', margin: '0 0 20px', fontSize: '0.9rem' }}>Instant scheduling with
1000+ approved medical specialists across specialties.</p>
                    <div style={{ display: 'flex', justifyContent: 'space-around', borderTop: '1px solid
rgba(255,255,255,0.05)', paddingTop: '20px' }}>
                      <div>
                        <h4 style={{ margin: 0, color: '#ef4444' }}>1k+</h4>
                        <span style={{ fontSize: '0.75rem', color: '#808090' }}>Doctors</span>
                      </div>
                      <div>
                        <h4 style={{ margin: 0, color: '#ef4444' }}>24/7</h4>
                        <span style={{ fontSize: '0.75rem', color: '#808090' }}>Support</span>
                      </div>
                      <div>
                        <h4 style={{ margin: 0, color: '#ef4444' }}>100%</h4>
                        <span style={{ fontSize: '0.75rem', color: '#808090' }}>Verified</span>
                      </div>
                    </div>
                  </div>
                </div>
              </section>

              {/* About Us Section */}
              <section className="about-section" style={{ background: '#111115', borderTop: '1px solid rgba(255,
255, 255, 0.08)', padding: '80px 40px' }}>
                <div style={{ maxWidth: '1200px', margin: '0 auto' }}>
                  <div style={{ textAlign: 'center', marginBottom: '50px' }}>
                    <h2 style={{ fontSize: '2rem', margin: '0 0 10px' }}>About MediCareBook</h2>
                    <p style={{ color: '#808090', maxWidth: '600px', margin: '0 auto' }}>An innovative healthcare
connection platform bridging the gap between patients, clinical providers, and administrators.</p>
                  </div>

                  <div style={{ display: 'flex', gap: '30px', flexWrap: 'wrap' }}>
                    <div style={{ flex: 1, minWidth: '250px', background: '#0a0a0c', border: '1px solid rgba(255, 255,
255, 0.05)', borderRadius: '12px', padding: '30px' }}>
                      <div style={{ fontSize: '2rem', marginBottom: '15px' }}>🔍</div>
                      <h3 style={{ margin: '0 0 10px', fontSize: '1.2rem' }}>Specialist Directory</h3>
                      <p style={{ color: '#808090', margin: 0, fontSize: '0.9rem', lineHeight: '1.5' }}>Search and filter
clinical specialists by specialization, location, timings, and consulting fees.</p>
                    </div>
                    <div style={{ flex: 1, minWidth: '250px', background: '#0a0a0c', border: '1px solid rgba(255, 255,
255, 0.05)', borderRadius: '12px', padding: '30px' }}>
                      <div style={{ fontSize: '2rem', marginBottom: '15px' }}>📅</div>
                      <h3 style={{ margin: '0 0 10px', fontSize: '1.2rem' }}>Easy Booking</h3>
                      <p style={{ color: '#808090', margin: 0, fontSize: '0.9rem', lineHeight: '1.5' }}>Choose preferred
dates and slots, and upload relevant medical history reports directly to the doctor's review queue.</p>
                    </div>
                    <div style={{ flex: 1, minWidth: '250px', background: '#0a0a0c', border: '1px solid rgba(255, 255,
255, 0.05)', borderRadius: '12px', padding: '30px' }}>
                      <div style={{ fontSize: '2rem', marginBottom: '15px' }}>🔒</div>
                      <h3 style={{ margin: '0 0 10px', fontSize: '1.2rem' }}>Role-Based Access</h3>
                      <p style={{ color: '#808090', margin: 0, fontSize: '0.9rem', lineHeight: '1.5' }}>Verified
platforms logins with role-based JWT tokens, password hashing, and admin approval workflows.</p>
                    </div>
                  </div>
                </div>
              </section>
```

```
97              {/* Footer */}
98              <footer style={{ padding: '30px 40px', background: '#0a0a0c', borderTop: '1px solid rgba(255, 255, 255,
99          0.05)', textAlign: 'center', color: '#808090', fontSize: '0.85rem' }}>
100                 <p style={{ margin: 0 }}>&copy; 2026 MediCareBook Inc. All rights reserved.</p>
101             </footer>
102         </div>
        );
    };

    export default Home;
```

Source Code: frontend/src/pages/Register.jsx

```
1   import React, { useState } from 'react';
2   import { useNavigate, Link } from 'react-router-dom';
3   import api from '../services/api';
4   import '../styles/login.css'; // Reuse login styles
5
6   const Register = () => {
7     const navigate = useNavigate();
8     const [fullName, setFullName] = useState('');
9     const [email, setEmail] = useState('');
10    const [password, setPassword] = useState('');
11    const [phone, setPhone] = useState('');
12    const [type, setType] = useState('user'); // 'user' or 'admin'
13    const [error, setError] = useState('');
14    const [loading, setLoading] = useState(false);
15
16    const handleSubmit = async (e) => {
17      e.preventDefault();
18      setError('');
19
20      if (!fullName || !email || !password || !phone) {
21        setError('Please complete all registration fields');
22        return;
23      }
24
25      setLoading(true);
26      try {
27        const response = await api.post('/user/register', {
28          fullName,
29          email,
30          password,
31          phone,
32          type
33        });
34        if (response.data.success) {
35          navigate('/login');
36        } else {
37          setError(response.data.message || 'Registration failed');
38        }
39      } catch (err) {
40        setError(err.response?.data?.message || 'Registration failed');
```

```
41      } finally {
42        setLoading(false);
43      }
44    };
45
46    return (
47      <div className="login-page-body">
48        <header className="app-header" style={{ position: 'absolute', top: 0, left: 0, right: 0, background:
49  'transparent', backdropFilter: 'none', borderBottom: 'none' }}>
50          <div className="logo-group">
51            <div className="logo-icon">D</div>
52            <div className="logo-text" style={{ color: '#fff' }}>Doctors' Call</div>
53          </div>
54          <nav>
55            <Link to="/" style={{ color: '#fff', textDecoration: 'none', marginRight: '20px', fontWeight: 'bold'
56  }}>Home</Link>
57            <Link to="/login" style={{ color: '#fff', textDecoration: 'none', marginRight: '20px', fontWeight:
58  'bold' }}>Login</Link>
59            <Link to="/register" style={{ color: '#ef4444', textDecoration: 'none', fontWeight: 'bold'
```

```
}}>Register</Link>
            </nav>
          </header>

          <div className="login-wrapper" style={{ marginTop: '80px' }}>
            <div className="brand-section">
              <div className="logo-group">
                <div className="logo-icon">D</div>
                <div className="logo-text">Doctors' Call</div>
              </div>

              <div className="brand-details">
                <h1>Create an Account</h1>
                <p>Join the healthcare network. Register as a patient to schedule consultations, or apply as a
provider to host tele-presence sessions.</p>
              </div>

              <div className="brand-footer">
                &copy; 2026 PTR MEDICAL SYSTEMS. ALL RIGHTS RESERVED.
              </div>
            </div>

            <div className="form-section">
              <div>
                <div className="form-header">
                  <h2>User Registration</h2>
```

```
                  <p>Register credentials to initialize clinical alignment</p>
                </div>

                <form onSubmit={handleSubmit}>
                  <div className="input-group" style={{ marginBottom: '15px' }}>
                    <label htmlFor="fullName">Full Name</label>
                    <div className="input-wrapper">
                      <input
                        type="text"
                        id="fullName"
                        required
                        placeholder="John Doe"
                        value={fullName}
                        onChange={(e) => setFullName(e.target.value)}
                      />
                    </div>
                  </div>

                  <div className="input-group" style={{ marginBottom: '15px' }}>
                    <label htmlFor="email">Email Address</label>
                    <div className="input-wrapper">
                      <input
                        type="email"
                        id="email"
                        required
                        placeholder="johndoe@email.com"
                        value={email}
                        onChange={(e) => setEmail(e.target.value)}
                      />
                    </div>
                  </div>

                  <div className="input-group" style={{ marginBottom: '15px' }}>
                    <label htmlFor="password">Password</label>
                    <div className="input-wrapper">
                      <input
```

```
                                type="password"
                                id="password"
                                required
                                placeholder="••••••••"
```

```
                                value={password}
                                onChange={(e) => setPassword(e.target.value)}
                            />
                        </div>
                    </div>

                    <div className="input-group" style={{ marginBottom: '15px' }}>
                        <label htmlFor="phone">Phone Number</label>
                        <div className="input-wrapper">
                            <input
                                type="text"
                                id="phone"
                                required
                                placeholder="9998887776"
                                value={phone}
                                onChange={(e) => setPhone(e.target.value)}
                            />
                        </div>
                    </div>

                    <div className="input-group" style={{ marginBottom: '15px' }}>
                        <label htmlFor="type">Account Role</label>
                        <div className="input-wrapper">
                            <select
                                id="type"
                                required
                                value={type}
                                onChange={(e) => setType(e.target.value)}
                            >
                                <option value="user" style={{ background: '#1c1c1e' }}>User (Patient)</option>
                                <option value="admin" style={{ background: '#1c1c1e' }}>Admin</option>
                            </select>
                        </div>
                        <div className={`error-message ${error ? 'visible' : ''}`}>{error}</div>
                    </div>

                    <button type="submit" className="btn-submit" disabled={loading}>
                        <span>REGISTER</span>
                        {loading && <div className="loader" style={{ display: 'block' }}></div>}
                    </button>
```

```

                    <div className="toggle-link">
                        Already have an account? <Link to="/login">Login</Link>
                    </div>
                </form>
            </div>
        </div>
    </div>
  );
};

export default Register;

```

Source Code: frontend/src/pages/Profile.jsx

```
import React, { useState, useEffect } from 'react';
import { useNavigate, Link } from 'react-router-dom';
import api from '../services/api';
import '../styles/profile.css';

const Profile = () => {
    const navigate = useNavigate();
    const [user, setUser] = useState(null);
    const [loading, setLoading] = useState(true);

    useEffect(() => {
        const fetchProfile = async () => {
            try {
                const res = await api.get('/auth/profile');
                setUser(res.data);
            } catch (err) {
                console.error('Failed to fetch user profile:', err);
                // Redirect to login if token invalid
                localStorage.removeItem('userInfo');
                navigate('/login');
            } finally {
                setLoading(false);
            }
        };
        fetchProfile();
    }, [navigate]);

    const handleLogout = () => {
        localStorage.removeItem('userInfo');
        navigate('/login');
    };

    if (loading) {
        return (
            <div className="profile-page-body" style={{ justifyContent: 'center', alignItems: 'center', display: 'flex'
}}>
                <h3>Loading clinician profile...</h3>
            </div>
        );
    }
```

```
    if (!user) return null;

    // Get initials for profile avatar
    const initials = user.email.substring(0, 2).toUpperCase();

    return (
        <div className="profile-page-body">
            <header className="app-header">
                <div className="logo-group">
                    <div className="logo-icon" style={{ width: '28px', height: '28px', background: 'var(--accent-red)',
borderRadius: '6px', display: 'flex', justifyContent: 'center', alignItems: 'center', fontWeight: 700, color: '#ffffff'
}}>D</div>
                    <div className="logo-text">Doctors' Call</div>
                </div>
                <nav>
                    <Link to="/scanner">Precision Anatomy Portal</Link>
                    <Link to="/hub">Doctors' Hub</Link>
                    <Link to="/session">Session</Link>
```

```
59                  <Link to="/profile" className="active">Profile</Link>
60                  <a href="#" onClick={(e) => { e.preventDefault(); handleLogout(); }}>Logout</a>
61                </nav>
62              </header>
63
64              <div className="profile-container">
65                {/* Profile Details Panel (Left) */}
66                <div className="profile-details-card">
67                  <div className="profile-header">
68                    <div className="profile-avatar">{initials}</div>
69                    <div className="profile-title">
70                      <h2>Clinician Diagnostic Profile</h2>
71                      <p>Authorized access matrix credentials</p>
72                    </div>
73                  </div>
74
75                  <div className="profile-grid">
76                    <div className="grid-item">
77                      <div className="grid-item-label">Account ID / Email</div>
78                      <div className="grid-item-value">{user.email}</div>
79                    </div>
80                    <div className="grid-item">
                        <div className="grid-item-label">Insurance ID</div>
                        <div className="grid-item-value">{user.insuranceId || 'None Registered'}</div>
```

```
81                    </div>
82                    <div className="grid-item">
83                      <div className="grid-item-label">Age</div>
84                      <div className="grid-item-value">{user.age || '—'} years</div>
85                    </div>
86                    <div className="grid-item">
87                      <div className="grid-item-label">Gender</div>
88                      <div className="grid-item-value" style={{ textTransform: 'capitalize' }}>{user.gender || '—
89  '}</div>
90                    </div>
91                    <div className="grid-item">
92                      <div className="grid-item-label">Blood Group</div>
93                      <div className="grid-item-value">{user.bloodGroup || '—'}</div>
94                    </div>
95                    <div className="grid-item">
96                      <div className="grid-item-label">Reported Health Condition</div>
97                      <div className="grid-item-value">{user.healthCondition || 'None'}</div>
98                    </div>
99                  </div>
100
101                 <div className="saved-parts-section">
102                   <h3>Saved Telemetry Anatomy</h3>
103                   {user.savedParts && user.savedParts.length > 0 ? (
104                     <div className="parts-list">
105                       {user.savedParts.map(part => (
106                         <span key={part} className="part-pill">{part}</span>
107                       ))}
108                     </div>
109                   ) : (
110                     <p style={{ color: 'var(--text-secondary)', fontSize: '0.85rem' }}>No anatomical matrix segments
111 saved yet.</p>
112                   )}
113                 </div>
114               </div>
115
116               {/* Dashboard Nav Actions Panel (Right) */}
117               <div className="profile-nav-actions">
118                 <div className="nav-button-card" onClick={() => navigate('/hub')}>
```

```
                    <div className="nav-card-icon">⛨</div>
                    <div className="nav-card-title">Hire Doctor Marketplace</div>
                    <div className="nav-card-desc">Search, filter, and book online/offline consultation matrices with medical specialists.</div>
                </div>
```

```

                <div className="nav-button-card" onClick={() => navigate('/scanner')}>
                    <div className="nav-card-icon">⚡</div>
                    <div className="nav-card-title">Precision Anatomy Scanner</div>
                    <div className="nav-card-desc">Scan myofibrils, skeletal arrays, and visceral organs to extract real-time diagnostic telemetry.</div>
                </div>
            </div>
        </div>
    </div>
    );
};

export default Profile;
```

Source Code: frontend/src/pages/Session.jsx

```
1   import React, { useState, useEffect } from 'react';
2   import { Link, useNavigate } from 'react-router-dom';
3   import api from '../services/api';
4   import '../styles/session.css';
5
6   const Session = () => {
7     const navigate = useNavigate();
8     const [sessions, setSessions] = useState([]);
9     const [loading, setLoading] = useState(true);
10    const [currentTime, setCurrentTime] = useState(new Date());
11
12    // Fetch user appointments/sessions
13    const fetchSessions = async () => {
14      try {
15        const res = await api.get('/appointments/my');
16        // Filter active sessions:
17        // 1. Pending admin approval
18        // 2. Approved but unpaid
19        // 3. Paid (confirmed), not expired, and calls remaining > 0
20        const active = res.data.filter(s => {
21          if (s.status === 'pending' || s.status === 'approved') {
22            return true;
23          }
24          if (s.status === 'confirmed') {
25            const expires = new Date(s.expiresAt);
26            const hasCalls = s.callsRemaining > 0;
27            return expires > new Date() && hasCalls;
28          }
29          return false;
30        });
31        setSessions(active);
32      } catch (err) {
33        console.error('Failed to fetch sessions', err);
34      } finally {
35        setLoading(false);
36      }
37    };
38
39    useEffect(() => {
40      fetchSessions();
```

```
41
42      // Update current time every minute to refresh countdowns
43      const timer = setInterval(() => {
44        setCurrentTime(new Date());
45      }, 60000);
46
47      return () => clearInterval(timer);
48    }, []);
49
50    const handleLogout = () => {
51      localStorage.removeItem('token');
52      localStorage.removeItem('userInfo');
53      navigate('/login');
54    };
55
56    const handleActionClick = async (session) => {
57      if (session.status === 'approved') {
58        // Redirect to purchase/checkout for this specific appointment
59        navigate('/purchase', {
```

```
60                 state: {
61                     appointmentId: session._id,
62                     doctor: session.doctorId
63                 }
64             });
65             return;
66         }
67
68         if (session.consultationType === 'online') {
69             try {
70                 // Decrement the calls remaining via the backend endpoint
71                 await api.post(`/appointments/${session._id}/use-call`);
72                 // Navigate to the video call session
73                 navigate('/call', {
74                     state: {
75                         doctor: session.doctorId,
76                         consultationType: 'online'
77                     }
78                 });
79             } catch (err) {
80                 alert(err.response?.data?.message || 'Failed to initiate video call.');
```

```
81                 fetchSessions(); // Refresh to reflect correct state
82             }
83         } else {
84             try {
85                 await api.post(`/appointments/${session._id}/use-call`);
86                 navigate('/call', {
87                     state: {
88                         doctor: session.doctorId,
89                         consultationType: 'offline'
90                     }
91                 });
92             } catch (err) {
93                 alert(err.response?.data?.message || 'Failed to open location navigation.');
94                 fetchSessions();
95             }
96         }
97     };
98
99     // Calculate time remaining in hours and minutes
100    const getTimeRemainingText = (session) => {
101        if (session.status === 'pending') {
102            return 'Awaiting Approval';
103        }
104        if (session.status === 'approved') {
105            return 'Awaiting Payment';
106        }
107        const expiry = new Date(session.expiresAt);
108        const diffMs = expiry - currentTime;
109        if (diffMs <= 0) return 'Expired';
110
111        const diffHrs = Math.floor(diffMs / (1000 * 60 * 60));
112        const diffMins = Math.floor((diffMs % (1000 * 60 * 60)) / (1000 * 60));
113
114        if (diffHrs > 0) {
115            return `Expires in ${diffHrs}h ${diffMins}m`;
116        }
117        return `Expires in ${diffMins}m`;
118    };
119
120    return (
```

```
121         <div className="session-page-body">
122           <header className="app-header">
123             <div className="logo-group">
124               <div className="logo-icon">D</div>
125               <div className="logo-text">Doctors' Call</div>
126             </div>
127             <nav>
128               <Link to="/scanner">Precision Anatomy Portal</Link>
129               <Link to="/hub">Doctors' Hub</Link>
130               <Link to="/session" className="active">Session</Link>
131               <Link to="/profile">Profile</Link>
132               <a href="#" onClick={(e) => { e.preventDefault(); handleLogout(); }}>Logout</a>
133             </nav>
134           </header>
135
136           <div className="session-container">
137             <div className="session-title-block">
138               <h2>Authorized Consultation Sessions</h2>
139               <p>Track your active medical connection windows, video consult pathways, and navigation
140 vectors.</p>
141             </div>
142
143             {loading ? (
144               <div style={{ textAlign: 'center', padding: '40px' }}>
145                 <h3>Iterating secure sessions database...</h3>
146               </div>
147             ) : sessions.length === 0 ? (
148               <div className="no-sessions-card">
149                 <h3>No Active Sessions Available</h3>
150                 <p>You currently do not have any pending, approved, or paid consultation matrices. Visit the
151 Doctors' Hub to authorize an appointment slot.</p>
152                 <button className="btn-action" style={{ maxWidth: '200px' }} onClick={() => navigate('/hub')}>
153                   GO TO HUB
154                 </button>
155               </div>
156             ) : (
157               <div className="sessions-list">
158                 {sessions.map(session => (
159                   <div className="session-card" key={session._id}>
160                     <div className="session-info">
                        <div className="session-header-row">
                          <span className={`session-type-badge ${session.consultationType}`}>
```

```
161                             {session.consultationType === 'online' ? 'Online Video' : 'Offline Clinic'}
162                           </span>
163                           <span
164                             className="session-expiry-badge"
165                             style={{
166                               background: session.status === 'pending' ? 'rgba(245, 158, 11, 0.1)' : session.status
167 === 'approved' ? 'rgba(59, 130, 246, 0.1)' : 'rgba(239, 68, 68, 0.1)',
168                               color: session.status === 'pending' ? '#f59e0b' : session.status === 'approved' ?
169 '#3b82f6' : 'var(--accent-red)'
170                             }}
171                           >
172                             {getTimeRemainingText(session)}
173                           </span>
174                         </div>
175                         <div className="session-doc-name">{session.doctorId?.name || 'Dr. Medical
176 Expert'}</div>
177                         <div className="session-doc-specialty">{session.doctorId?.specialty || 'General
178 Practitioner'}</div>
```

```
                        <div className="session-meta-row">
                          <div className="session-meta-item">
                            🏥 <span>{session.doctorId?.hospitalName || 'Clinic'}</span>
                          </div>
                          <div className="session-meta-item">
                            📍 <span>{session.doctorId?.place || 'Location'}</span>
                          </div>
                        </div>
                      </div>

                      <div className="session-actions">
                        <div className="calls-remaining-text">
                          {session.status === 'pending' ? (
                            <span style={{ color: '#f59e0b' }}>Awaiting Admin Approval</span>
                          ) : session.status === 'approved' ? (
                            <span style={{ color: '#3b82f6' }}>Approved by Admin (Unpaid)</span>
                          ) : (
                            <>
                              Remaining authorizations:{' '}
                              <span className="calls-remaining-highlight">
                                {session.callsRemaining} / 2
                              </span>
                            </>
                          )}
                        </div>
```

```
                        <button
                          className="btn-action"
                          disabled={session.status === 'pending'}
                          style={{
                            background: session.status === 'pending' ? 'rgba(255,255,255,0.05)' : session.status
=== 'approved' ? '#3b82f6' : 'var(--accent-red)',
                            color: session.status === 'pending' ? 'var(--text-secondary)' : '#ffffff',
                            cursor: session.status === 'pending' ? 'not-allowed' : 'pointer'
                          }}
                          onClick={() => handleActionClick(session)}
                        >
                          {session.status === 'pending'
                            ? 'AWAITING APPROVAL'
                            : session.status === 'approved'
                              ? 'PAY NOW'
                              : session.consultationType === 'online'
                                ? 'START CALL'
                                : 'SEE MAP & NAVIGATE'
                          }
                        </button>
                      </div>
                    </div>
                  ))}
                </div>
              )}
            </div>
          </div>
        );
};

export default Session;
```

Source Code: frontend/src/pages/Call.jsx

```
1   import React, { useState, useEffect, useRef } from 'react';
2   import { useLocation, useNavigate, Link } from 'react-router-dom';
3   import '../styles/call.css';
4
5   const cityCoordinates = {
6     'Delhi': 'bbox=77.10%2C28.55%2C77.30%2C28.75&layer=mapnik',
7     'Mumbai': 'bbox=72.75%2C18.90%2C72.95%2C19.10&layer=mapnik',
8     'Pune': 'bbox=73.75%2C18.45%2C73.95%2C18.65&layer=mapnik',
9     'Bangalore': 'bbox=77.50%2C12.85%2C77.70%2C13.05&layer=mapnik',
10    'Hyderabad': 'bbox=78.35%2C17.30%2C78.55%2C17.50&layer=mapnik',
11    'Ahmedabad': 'bbox=72.50%2C22.95%2C72.70%2C23.15&layer=mapnik',
12    'Kolkata': 'bbox=88.30%2C22.50%2C88.50%2C22.70&layer=mapnik'
13  };
14
15  const directionSteps = {
16    'Delhi': [
17      'Exit Metro station gate 3 and turn right towards Ring Road.',
18      'Proceed straight for 500 meters, past the central park entrance.',
19      'The clinic is situated inside Apollo Medical Centre on the 2nd Floor.'
20    ],
21    'Mumbai': [
22      'Head south on Western Express Highway past the flyover.',
23      'Take the exit towards Bandra Kurla Complex (BKC).',
24      'Drive 300 meters, the diagnostic facility is inside building C on your left.'
25    ],
26    'Pune': [
27      'Drive down Senapati Bapat Road towards University circle.',
28      'Turn left at the junction after the mall.',
29      'The clinic is situated on the 3rd Floor of the Apex Care Building.'
30    ],
31    'Bangalore': [
32      'From Outer Ring Road, head towards Sarjapur Road junction.',
33      'Take a U-turn after the business park complex.',
34      'Look for KIMS Outpost building next to the central library.'
35    ],
36    'Hyderabad': [
37      'Proceed past HITEC City metro pillars towards Mindspace junction.',
38      'Turn left at the cyber gateway park side-road.',
39      'Enter Global Medical Hub tower A, clinic is on suite 402.'
40    ],
```

```
41    'Ahmedabad': [
42      'Drive down SG Highway past the main bridge road.',
43      'Take the service road towards the corporate sector.',
44      'Clinic is on ground floor of the newly built AIMS Plaza.'
45    ],
46    'Kolkata': [
47      'Head towards Salt Lake sector V main bypass road.',
48      'Turn right past the IT park office tower block.',
49      'The facility is located on Fortis Care center level 1.'
50    ]
51  };
52
53  const Call = () => {
54    const location = useLocation();
55    const navigate = useNavigate();
56    const { doctor, consultationType = 'online' } = location.state || {};
57
58    const [isMuted, setIsMuted] = useState(false);
59    const [isVideoMuted, setIsVideoMuted] = useState(false);
```

```
    const [stream, setStream] = useState(null);
    const [mediaError, setMediaError] = useState('');

    const localVideoRef = useRef(null);

    // Initialize Camera and Microphone Stream
    useEffect(() => {
        if (consultationType === 'online') {
            const getMedia = async () => {
                try {
                    const mediaStream = await navigator.mediaDevices.getUserMedia({
                        video: true,
                        audio: true
                    });
                    setStream(mediaStream);
                    if (localVideoRef.current) {
                        localVideoRef.current.srcObject = mediaStream;
                    }
                } catch (err) {
                    console.error('Camera/Mic permission failed:', err);
                    setMediaError('Unable to access camera or microphone. Please verify site permissions.');
```

```
                }
            };
            getMedia();
        }

        // Clean up media tracks when leaving page
        return () => {
            if (stream) {
                stream.getTracks().forEach(track => track.stop());
            }
        };
    }, [consultationType]);

    const handleToggleMute = () => {
        if (stream) {
            const audioTrack = stream.getAudioTracks()[0];
            if (audioTrack) {
                audioTrack.enabled = !audioTrack.enabled;
                setIsMuted(!audioTrack.enabled);
            }
        }
    };

    const handleToggleVideo = () => {
        if (stream) {
            const videoTrack = stream.getVideoTracks()[0];
            if (videoTrack) {
                videoTrack.enabled = !videoTrack.enabled;
                setIsVideoMuted(!videoTrack.enabled);
            }
        }
    };

    const handleEndCall = () => {
        if (stream) {
            stream.getTracks().forEach(track => track.stop());
        }
        navigate('/hub');
    };

```

```
    if (!doctor) {
        return (
            <div className="call-page-body" style={{ justifyContent: 'center', alignItems: 'center', display: 'flex',
flexDirection: 'column', gap: '20px' }}>
                <h2>No Session Matrix Selected</h2>
                <Link to="/hub" style={{ color: 'var(--accent-red)', fontWeight: 'bold' }}>Return to Doctors'
Hub</Link>
            </div>
        );
    }

    const mapBbox = cityCoordinates[doctor.place] || cityCoordinates['Mumbai'];
    const steps = directionSteps[doctor.place] || directionSteps['Mumbai'];

    return (
        <div className="call-page-body">
            <header className="app-header">
                <div className="logo-group">
                    <div className="logo-icon">D</div>
                    <div className="logo-text">Doctors' Call</div>
                </div>
                <nav>
                    <Link to="/scanner">Precision Anatomy Portal</Link>
                    <Link to="/hub">Doctors' Hub</Link>
                    <Link to="/session">Session</Link>
                    <Link to="/profile">Profile</Link>
                    <a href="#" onClick={(e) => { e.preventDefault(); localStorage.removeItem('token');
navigate('/login'); }}>Logout</a>
                </nav>
            </header>

            <div className="call-container">
                <div style={{ width: '100%' }}>
                    <h2 style={{ fontSize: '1.6rem', fontWeight: 'bold' }}>
                        {consultationType === 'online' ? 'Active Consultation Matrix' : 'Offline Navigation Portal'}
                    </h2>
                    <p style={{ color: 'var(--text-secondary)', fontSize: '0.9rem', marginTop: '4px' }}>
                        Consulting {doctor.name} ({doctor.specialty})
                    </p>
                </div>

                <div className="consultation-panel">
                    {/* Tele-Presence Video Panel */}
```

```
                    {consultationType === 'online' ? (
                        <div className="video-section">
                            {/* Doctor Avatar Stream */}
                            <div className="doctor-stream" style={{ display: 'flex', flexDirection: 'column', justifyContent:
'center', alignItems: 'center', gap: '15px' }}>
                                <div style={{
                                    width: '120px',
                                    height: '120px',
                                    borderRadius: '50%',
                                    background: 'rgba(239, 68, 68, 0.1)',
                                    border: '2px solid var(--accent-red)',
                                    display: 'flex',
                                    justifyContent: 'center',
                                    alignItems: 'center',
                                    fontSize: '2.5rem',
                                    fontWeight: 'bold',
                                    color: 'var(--accent-red)'
```

```
                        }}>
                            {doctor.initials}
                        </div>
                        <div style={{ fontSize: '1.1rem', fontWeight: '600' }}>{doctor.name}</div>
                        <div style={{ fontSize: '0.85rem', color: 'var(--text-secondary)' }}>Connecting tele-presence stream...</div>
                    </div>

                    {/* Local Camera Stream */}
                    <div className="local-stream-container">
                        {mediaError ? (
                            <div style={{ fontSize: '0.7rem', padding: '10px', color: 'var(--accent-red)', textAlign: 'center' }}>
                                {mediaError}
                            </div>
                        ) : (
                            <video
                                ref={localVideoRef}
                                className="local-video"
                                autoPlay
                                playsInline
                                muted
                            />
                        )}
                    </div>
```

```
                    {/* Call Controls */}
                    <div className="call-controls">
                        <button
                            className={`control-btn mute-audio ${isMuted ? 'muted' : ''}`}
                            onClick={handleToggleMute}
                            title={isMuted ? 'Unmute Audio' : 'Mute Audio'}
                        >
                            🎙
                        </button>
                        <button
                            className={`control-btn mute-video ${isVideoMuted ? 'muted' : ''}`}
                            onClick={handleToggleVideo}
                            title={isVideoMuted ? 'Start Camera' : 'Stop Camera'}
                        >
                            📷
                        </button>
                        <button
                            className="control-btn end-call"
                            onClick={handleEndCall}
                            title="End Consultation"
                        >
                            ⏹
                        </button>
                    </div>
                </div>
            ) : (
                <div className="video-section" style={{ background: '#111', display: 'flex', flexDirection: 'column', gap: '10px', padding: '20px' }}>
                    <div style={{ fontSize: '1.2rem', fontWeight: 'bold', color: 'var(--accent-red)' }}>Clinic Consultation Protocol</div>
                    <p style={{ fontSize: '0.9rem', color: 'var(--text-secondary)', textAlign: 'center', maxWidth: '400px' }}>
                        You have selected an offline consultation slot. Please use the navigation coordinates in the side panel to proceed to {doctor.hospitalName || 'the clinic'}.
                    </p>
```

```
                        <button
                            onClick={() => navigate('/hub')}
                            style={{ marginTop: '20px', padding: '12px 24px', background: 'var(--accent-red)', color:
'#ffffff', fontWeight: 'bold', border: 'none', borderRadius: '8px', cursor: 'pointer' }}
                        >
                            BACK TO HUB
                        </button>
                    </div>
                )}
```

```
                {/* Navigation Map Panel */}
                <div className="map-section">
                    <h3 style={{ fontSize: '1rem', textTransform: 'uppercase', letterSpacing: '1px', color: 'var(--accent-
red)' }}>
                        Clinic Location Map
                    </h3>

                    <div className="map-frame-container">
                        <iframe
                            title="Clinic Location Map"
                            width="100%"
                            height="100%"
                            frameBorder="0"
                            scrolling="no"
                            marginHeight="0"
                            marginWidth="0"
                            src={`https://www.openstreetmap.org/export/embed.html?${mapBbox}`}
                            style={{ border: 'none' }}
                        />
                    </div>

                    <div className="directions-box">
                        <h4 style={{ fontSize: '0.85rem', fontWeight: 'bold', marginBottom: '10px', color: 'var(--accent-
amber)' }}>
                            Route Guidance ({doctor.place})
                        </h4>
                        {steps.map((step, idx) => (
                            <div className="direction-step" key={idx}>
                                <div className="step-num">{idx + 1}</div>
                                <div>{step}</div>
                            </div>
                        ))}
                    </div>
                </div>
            </div>
        </div>
    );
};

export default Call;
```